FOODONIX - A Food Delivery Web Application

A project Report submitted to

GURU GHASIDAS VISHWAVIDYALAYA, BILASPUR

Session: 2025-2026

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

MASTER OF COMPUTER APPLICATION

By

ANKIT ARPAN DAS

Roll No.:24072110

Enrolment No.: GGV/24/05010

SEMESTER: M.C.A.- IV

Under the Guidance of

DR. RATNESH PRASAD SRIVASTAVA

(Associate Professor Dept. of CSIT)

GURU GHASIDAS VISHWAVIDYALAYA, BILASPUR

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Session:

2025-26

CERTIFICATE OF GUIDE

This is to certify that the work incorporated in the project "**Foodonix-Food delivery web application"** is a record of six-month project work assigned by our Institution, successfully carried out by **ANKIT ARPAN DAS** bearing Enrolment No- GGV/24/05010 under my guidance and supervision for the award of Degree of **Master of Computer Application** of **DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY, GURU GHASIDAS VISHWAVIDYALAYA, BILASPUR** (**C.G.), INDIA**. To the best of my knowledge and belief the report embodies the work of the candidate him/herself and has duly been successfully completed.

Signature of the Guide	**Signature of HOD**
Dr. Ratnesh Prasad Srivastava	Dr. Pushpalata Pujari
(Associate Professor, Dept. of CSIT)	(Associate Professor Dept. of CSIT)

GURU GHASIDAS VISHWAVIDYALAYA, BILASPUR

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Session:

2025-26

DELCLARATION BY THE CANDIDATE

I, **ANKIT ARPAN DAS,** Student of IV Semester MCA, **DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY, GURU GHASIDAS VISHWAVIDYALAYA, BILASPUR**, bearing Enrolment Number **GGV/24/05010** hereby declare that the project titled "**Foodonix-Food delivery web application**" has been carried out by me under the Guidance of **Mr. Ratnesh Prasad Srivastava** submitted in partial fulfilment of the requirements for the award of the Degree of Master of Computer Application by the Department Of Computer Science & Information Technology, Guru Ghasidas Vishwavidyalaya, Bilaspur during the academic year 2025-26. This report has not been submitted to any other Organization/University for any award of Degree/Diploma.

Date:

Place: Bilaspur

ANKIT ARPAN DAS

DEPT. of CSIT (MCA)

GURU GHASIDAS VISHWAVIDYALAYA

GURU GHASIDAS VISHWAVIDYALAYA, BILASPUR

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Session:

2025-26

CERTIFICATE BY THE EXAMINER

This is to certified that the project work entitled "**Foodonix-Food delivery web application** " submitted by **ANKIT ARPAN DAS** has completed under the guidance of Dr. **RATNESH PRASAD SRIVASTAVA** Dept. of "Computer Science & Information Technology**", GURU GHASIDAS VISHWAVIDYALAYA Bilaspur (C.G.)** has been examined by the undersigned as a part of the examination for the award of the M.Sc. Degree in Dept. of "Computer Science & Information Technology" **in GURU GHASIDAS CENTRAL UNIVERSITY BILASPUR (C.G.)**

"Project Examined & Approved"

---------- **----------**

Internal Examiner **External Examiner**

Date: **Date:**

Signature of HOD(CSIT)

ABSTRACT

The rapid proliferation of the internet and mobile technology has transformed almost every aspect of modern life, including the way people order and consume food. The traditional method of placing food orders — either by visiting a restaurant in person or making a telephone call — has significant limitations in terms of convenience, speed, and scalability. To address these challenges, this project presents **Foodonix**, a fully functional, web-based Online Food Delivery System developed using Java Spring Boot, JSP (JavaServer Pages), MySQL, and Maven.

Foodonix is designed to bridge the gap between customers and restaurants through an intuitive digital platform. The system supports three distinct user roles: **Customer**, **Restaurant Admin**, and **System Administrator**. Customers can browse a comprehensive list of restaurants, view detailed menus with pricing, nutritional information, and food-type classifications (Vegetarian/Non-Vegetarian), add items to a dynamic shopping cart, apply discount coupons, and place orders using multiple payment methods including Cash-on-Delivery (COD) and Online Payment. The system tracks orders in real time and maintains a complete order history for each user.

From an administrative perspective, the system provides robust tools for managing restaurants, menu items, food categories, and user accounts. Administrators can update the status of orders, manage active/inactive listings, and oversee the overall platform operations through a dedicated admin dashboard.

The architecture of Foodonix follows the **Model-View-Controller (MVC)** design pattern, which ensures a clean separation of concerns, high maintainability, and ease of scalability. The back-end leverages Spring Data JPA with Hibernate ORM for database interactions with a normalized MySQL relational database. The front-end is built using JSP templates with Bootstrap 5 for a responsive, mobile-friendly user interface.

This report documents the complete software development lifecycle of the Foodonix project, including requirement analysis, system design, UML diagrams, database schema, implementation details with code snippets, testing results, and future enhancement possibilities. The project demonstrates practical application of core software engineering principles and full-stack Java web development skills.

Keywords: Online Food Delivery, Spring Boot, MVC Architecture, JSP, MySQL, JPA Hibernate, E-Commerce, Web Application, Foodonix.

TABLE OF CONTENTS

Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Chapter 1 Introduction	1-3
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Project	2
1.4 Scope of the Project	3
Chapter 2 Literature Review	4-6
2.1 Overview of Existing Food Delivery Platforms	4
2.2 Comparative Analysis of Existing Systems	5
2.3 Research Gap and Motivation	5
Chapter 3 System Analysis	7-10
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	8
3.3 Feasibility Study	9
Chapter 4 System Design	11-18
4.1 System Architecture	11
4.2 Use Case Diagram	12
4.3 Class Diagram	12
4.4 Sequence Diagrams	13
4.5 Activity Diagram	14
4.6 Database Design (ER Diagram)	15
4.7 Database Tables	17
Chapter 5 Technology Stack	19-21
5.1 Programming Languages	19
5.2 Frameworks and Libraries	19
5.3 Tools and Software Requirements	20
5.4 Hardware Requirements	20
Chapter 6 Implementation	22-38
6.1 Project Structure	22
6.2 User Authentication Module	23
6.3 Restaurant and Menu Module	26

6.4 Shopping Cart Module 27
6.5 Order Management Module 30
6.6 Admin Module 32
6.7 Frontend (JSP Views) 33
6.8 Database Queries (SQL) 35

Chapter 7 Testing 39-42
7.1 Unit Testing 39
7.2 Integration Testing 39
7.3 System Testing 41
7.4 Test Cases and Results 42

Chapter 8 Results and Discussion 43

Chapter 9 Advantages and Limitations 55

Chapter 10 Future Enhancements 57

Chapter 11 Conclusion 59

References 61

CHAPTER-1
INTRODUCTION

The digital revolution has fundamentally reshaped consumer behavior across all industries. In the food and beverage sector, this transformation is most prominently visible through the rapid rise of online food delivery platforms. Applications like Zomato, Swiggy, Uber Eats, and DoorDash have not only changed how people interact with restaurants but have also created an entirely new ecosystem of commerce, logistics, and customer service.

India, in particular, has witnessed exponential growth in the food tech sector. With a young, tech-savvy population and increasing smartphone penetration, the online food delivery market in India is projected to reach USD 21.41 billion by 2026, growing at a CAGR of over 18%. This phenomenal growth underscores the need for robust, scalable, and user-friendly food delivery platforms.

Foodonix is designed as a comprehensive, end-to-end online food delivery solution that caters to the needs of all stakeholders: customers who want to order food with ease, restaurant owners who need to manage their digital presence efficiently, and platform administrators who oversee the overall operations. Built on the Java Spring Boot framework with a MySQL backend, Foodonix embodies the principles of modern web application development.

1.1 Background and Motivation

Traditional food ordering involved visiting a physical restaurant or calling by phone — both methods laden with inefficiencies. Long wait times, limited menu visibility, inability to track orders, and difficulty managing peak-hour demands are some of the common challenges faced by both consumers and restaurant operators in the conventional model.

The COVID-19 pandemic further accelerated the adoption of online food ordering, as lockdowns and social distancing norms made contactless delivery not just convenient but essential. Restaurants that previously relied exclusively on dine-in customers had to rapidly pivot to online delivery models to sustain their businesses.

This project was motivated by the desire to build a practical, real-world application that addresses these challenges. The developer, being a resident of Hyderabad — one of India's largest food markets — witnessed firsthand the need for efficient digital food ordering systems, particularly among smaller local restaurants that lack the resources to integrate with large commercial platforms.

Furthermore, from an academic perspective, this project provides an excellent opportunity to apply concepts learned throughout the B.Tech curriculum — including object-oriented programming, database management, web technologies, and software engineering principles — in a cohesive, real-world context.

1.2 Problem Statement

Despite the availability of large-scale food delivery platforms, several pain points remain unaddressed, particularly for smaller restaurants and local food businesses:

- Large platforms charge **high commission fees** (ranging from 15% to 30% per order), making them economically unviable for small restaurants.
- Existing solutions often require **complex API integrations** and significant technical expertise to operate.
- Many platforms lack adequate support for **regional languages and local food categories**, limiting their reach.
- The **order tracking** and **communication mechanism** between customers and restaurants is often inadequate on smaller platforms.
- There is a need for a **customizable, open, and lightweight** food delivery solution that restaurants can deploy independently.

The proposed system, Foodonix, aims to address these problems by providing a complete, self-contained food delivery web application that is easy to deploy, maintain, and scale. The system is designed to be adopted by institutions, small restaurant chains, or individual food businesses seeking a digital ordering platform without the overhead of third-party marketplace dependencies.

1.3 Objectives of the Project

The primary objectives of developing the Foodonix online food delivery system are as follows:

- To design and implement a **fully functional web-based food delivery platform** that supports end-to-end ordering workflows.
- To build a **multi-role authentication system** supporting Customers, Restaurant Admins, and System Administrators.
- To develop a **dynamic restaurant and menu management module** allowing admins to add, update, and deactivate menu items and restaurants in real time.
- To implement a **shopping cart system** with support for quantity adjustments, item removal, and real-time total calculation.
- To integrate a **discount and coupon management system** that applies percentage-based discounts to eligible orders.
- To support multiple **payment methods** including Cash-on-Delivery (COD) and Online Payment simulation.
- To provide a **comprehensive order tracking** system with status updates (Pending, Confirmed, Out for Delivery, Delivered).
- To design a **normalized relational database schema** using MySQL that ensures data integrity and efficient querying.

- To apply **MVC architectural principles** using Spring Boot to ensure code maintainability and separation of concerns.
- To create a **responsive, user-friendly front-end** using JSP and Bootstrap 5 that works across desktop and mobile devices.

1.4 Scope of the Project

The scope of the Foodonix project encompasses the design and development of a web application that facilitates food ordering between customers and restaurants. The following aspects fall within the scope of this project:

- **User Management:** Registration, login, profile management, and role-based access control for Customers and Admins.
- **Restaurant Management:** CRUD operations for restaurant profiles including name, description, address, and images.
- **Menu Management:** Addition and management of menu items with categories, pricing, offer pricing, food type (Veg/Non-Veg), and availability status.
- **Shopping Cart:** Dynamic cart with add, remove, and quantity update functionalities.
- **Order Processing:** Complete order lifecycle from cart checkout to order delivery with coupon code support.
- **Admin Dashboard:** Central interface for administrators to monitor and manage all platform activities.
- **Address Management:** Multiple delivery address support per customer.

The following aspects are **outside the scope** of this project:

- Real-time GPS-based delivery tracking and logistics management.
- Native mobile application (Android/iOS) development.
- Live payment gateway integration (only simulated in this version).
- AI-based food recommendation engine.
- Restaurant owner self-registration and approval workflow.

CHAPTER -2
LITERATURE REVIEW

The online food delivery domain has been extensively studied by researchers, industry analysts, and software engineers. This chapter reviews the existing systems, academic literature, and commercial platforms that have shaped the evolution of food delivery technology.

2.1 Overview of Existing Food Delivery Platforms

2.1.1 Zomato

Founded in 2008 and headquartered in Gurugram, India, Zomato is one of the largest food delivery platforms in India and internationally. The platform operates in over 500 cities and partners with more than 350,000 restaurants. Zomato uses a hyperlocal delivery model where orders are fulfilled by a network of independent delivery partners. Key features include real-time order tracking via GPS, user reviews and ratings, curated restaurant discovery, and subscription services (Zomato Gold/Pro). The system architecture employs microservices, Kafka for event streaming, and Elasticsearch for restaurant search. However, the high commission rates and stringent onboarding requirements limit accessibility for small restaurant owners.

2.1.2 Swiggy

Swiggy, founded in 2014 in Bangalore, is Zomato's primary competitor in the Indian market. It differentiates itself through its logistics-as-a-service model and guaranteed delivery time commitments. Swiggy's technology stack includes a robust recommendation engine, machine learning-based demand prediction, and dynamic pricing algorithms. Swiggy Instamart extended the platform beyond food to quick commerce (grocery delivery within 30 minutes). Academic studies on Swiggy's architecture highlight the use of Apache Spark for data processing and a distributed caching layer using Redis for performance optimization.

2.1.3 Uber Eats

Uber Eats, launched globally by Uber Technologies in 2014, leverages the same driver network as Uber's ride-hailing service. Its key differentiator is the integration with existing Uber infrastructure, enabling rapid market expansion. The system uses trip dispatch algorithms originally designed for ride-sharing, adapted for food delivery logistics. Uber Eats employs a GraphQL API layer for flexible data fetching across different client types (web, iOS, Android). The platform has faced regulatory challenges in several markets but remains a dominant player in North America and Europe.

2.1.4 Academic Research on Food Delivery Systems

Several academic papers have explored various aspects of food delivery systems. Kumar et al. (2021) proposed a machine learning-based demand forecasting model for food

delivery platforms that reduced delivery time variance by 23%. Sharma and Gupta (2022) analyzed user acceptance factors for food delivery applications in Indian tier-2 cities, finding that ease of use and perceived value were primary adoption drivers. Rahman et al. (2023) explored blockchain-based transparency mechanisms for food delivery supply chains, proposing a framework for verifiable order tracking.

2.1.5 Spring Boot-Based Web Applications in Research

The Spring Boot framework has been widely adopted in academic projects and industry applications for its convention-over-configuration approach, embedded server support, and comprehensive ecosystem. Several published projects in IEEE and ACM databases demonstrate the effectiveness of Spring Boot with JPA Hibernate for building data-intensive web applications. The combination of Spring MVC for request handling, Spring Data JPA for database abstraction, and JSP for server-side rendering represents a proven, battle-tested architecture for enterprise web applications.

2.2 Comparative Analysis of Existing Systems

Feature	Zomato	Swiggy	Uber Eats	Foodonix
Multi-restaurant	Yes	Yes	Yes	Yes
Real-time GPS Tracking	Yes	Yes	Yes	No (Scope)
Mobile App	iOS & Android	iOS & Android	iOS & Android	Web Only
Coupon/Discount	Yes	Yes	Yes	Yes
Admin Dashboard	Yes	Yes	Yes	Yes
Open Source	No	No	No	Yes
Tech Stack	Microservices	Microservices	GraphQL+Node	Spring Boot MVC
Deployment	Cloud (AWS)	Cloud (GCP)	Cloud (Azure)	Self-hosted
Commission Fees	15–30%	15–30%	15–30%	None
Customizability	Low	Low	Low	High

Table 2.1: Comparison of Existing Food Delivery Platforms

2.3 Research Gap and Motivation

The review of existing systems reveals several important gaps that this project aims to address:

- **Accessibility:** Major platforms are closed-source and require restaurants to accept high commission structures. There is a clear need for an open, customizable alternative.
- **Simplicity of Architecture:** Large-scale platforms use complex microservices architectures that are difficult to deploy and maintain for small teams. A monolithic MVC-based architecture, while less scalable, is far more suitable for educational projects and small-scale deployments.

- **Academic Value:** Most existing educational projects on food delivery focus on either the front-end or the back-end separately. Foodonix provides a complete, integrated implementation demonstrating full-stack development competency.
- **Local Market Needs:** The system is specifically designed keeping the Hyderabad food market in mind, with support for popular local food categories such as Biryani, South Indian, and regional sweets.

CHAPTER -3
SYSTEM ANALYSIS

System analysis is the process of studying a system, defining its components, and understanding how they interact to achieve the system's goals. This chapter details the requirements gathered through stakeholder interviews, use-case analysis, and domain research, along with a comprehensive feasibility study.

3.1 Functional Requirements

3.1.1 Customer Requirements

- **FR-C01:** The system shall allow new users to register by providing their first name, last name, email address, mobile number, and password.
- **FR-C02:** The system shall authenticate registered users via email and password combination.
- **FR-C03:** The system shall display a list of all active restaurants on the home page.
- **FR-C04:** The system shall allow customers to browse the complete menu of a selected restaurant, with items categorized by food type and category.
- **FR-C05:** The system shall allow customers to add menu items to a shopping cart with specified quantities.
- **FR-C06:** The system shall allow customers to update item quantities or remove items from the cart.
- **FR-C07:** The system shall allow customers to apply a coupon code during checkout to avail discounts.
- **FR-C08:** The system shall allow customers to select or add a delivery address during checkout.
- **FR-C09:** The system shall support payment via Cash-on-Delivery and Online Payment modes.
- **FR-C10:** The system shall generate a unique order number upon successful order placement.
- **FR-C11:** The system shall maintain a complete order history accessible to the customer.
- **FR-C12:** The system shall allow customers to view detailed information for each past order.
- **FR-C13:** The system shall allow customers to update their profile information.

3.1.2 Administrator Requirements

- **FR-A01:** The system shall provide a dedicated admin dashboard with summary statistics.
- **FR-A02:** The system shall allow admins to add, edit, and deactivate restaurant listings.

- **FR-A03:** The system shall allow admins to manage menu items for each restaurant, including setting offer prices.
- **FR-A04:** The system shall allow admins to create and manage food categories.
- **FR-A05:** The system shall allow admins to view all orders and update order statuses.
- **FR-A06:** The system shall allow admins to create and manage discount coupons.
- **FR-A07:** The system shall allow admins to manage user accounts.

Req. ID	Description	Priority	Role
FR-C01	User Registration	High	Customer
FR-C02	User Login/Logout	High	Customer
FR-C04	Browse Restaurant Menu	High	Customer
FR-C05	Add to Cart	High	Customer
FR-C07	Apply Coupon	Medium	Customer
FR-C09	Place Order	High	Customer
FR-A02	Manage Restaurants	High	Admin
FR-A03	Manage Menu Items	High	Admin
FR-A05	Manage Orders	High	Admin
FR-A06	Manage Coupons	Medium	Admin

Table 3.1: Functional Requirements Summary

3.2 Non-Functional Requirements

3.2.1 Performance

- The system shall load any page within 3 seconds under normal network conditions.
- The system shall be capable of handling at least 100 concurrent users without performance degradation.
- Database queries shall be optimized to return results within 500 milliseconds for standard operations.

3.2.2 Security

- All user passwords shall be stored securely. Production deployment should use BCrypt hashing (noted as enhancement).
- Session-based authentication shall prevent unauthorized access to protected pages.
- Role-based access control shall restrict admin functionalities from regular customers.
- Input validation shall be enforced on all form fields to prevent SQL injection and XSS attacks.

3.1.1 Usability

- The interface shall be intuitive and navigable without prior training.
- The system shall be responsive, supporting screen widths from 320px (mobile) to 1920px (desktop).
- Error messages shall be descriptive and guide the user toward corrective action.

3.1.2 Reliability

- The system shall maintain 99.5% uptime for production deployment.
- Database transactions shall be ACID-compliant to ensure data consistency.

Category	Requirement	Metric
Performance	Page Load Time	< 3 seconds
Performance	Concurrent Users	100+ users
Security	Session Timeout	30 minutes
Usability	Mobile Responsive	Yes (Bootstrap 5)
Reliability	Uptime	99.5%
Maintainability	MVC Architecture	Spring Boot MVC
Scalability	Database	MySQL (scalable)

Table 3.2: Non-Functional Requirements

3.2 Feasibility Study

3.2.1 Technical Feasibility

The technologies required to build Foodonix — Java, Spring Boot, MySQL, Maven, and JSP — are all mature, well-documented, and freely available. The Spring Boot framework simplifies the development of production-ready Spring applications by providing auto-configuration, embedded Tomcat server, and a rich ecosystem of starters. Java 17 LTS, used in this project, offers robust language features including records, sealed classes, and enhanced switch expressions. MySQL 8.0 provides excellent performance for the scale of this application. All development tools (Eclipse IDE, MySQL Workbench) are freely available. Therefore, the project is technically feasible without any significant barriers.

3.2.2 Economic Feasibility

The entire technology stack used in Foodonix is open-source and free of licensing costs. Development requires only a standard computer with at least 8GB RAM and a modern processor. Deployment can be done on any cloud provider (AWS Free Tier, Google Cloud Free Tier) or on a local server. The economic investment for this project is minimal and primarily consists of developer time. For a production environment, hosting costs would range from Rs. 500 to Rs. 2,000 per month depending on the traffic volume.

3.1.1 Operational Feasibility

The system is designed with ease of use as a primary consideration. The intuitive interface requires no technical training for end users. Restaurant administrators can manage their listings through simple form-based interfaces. System administrators have access to a centralized dashboard that provides a comprehensive view of all activities. The application follows standard web conventions, ensuring that users familiar with e-commerce platforms will find Foodonix immediately familiar and easy to navigate.

3.1.2 Schedule Feasibility

The project was completed in approximately 12 weeks, following a structured development schedule. The timeline was divided into four phases: Requirements Analysis and Design (Weeks 1–3), Backend Development (Weeks 4–7), Frontend Development (Weeks 8–10), and Testing and Documentation (Weeks 11–12). This schedule was realistic and achievable given the scope of the project and the available resources.

Feasibility Type	Assessment	Verdict
Technical	All technologies are mature and well-supported	Feasible
Economic	Zero licensing cost; minimal hardware	Feasible
Operational	Simple UI; standard web conventions	Feasible
Schedule	12-week development timeline	Feasible
Legal	All open-source; no proprietary components	Feasible

Table 3.3: Feasibility Analysis Matrix

CHAPTER -4
SYSTEM DESIGN

System design is the phase where theoretical requirements are translated into a concrete blueprint for the software system. This chapter presents the architectural design, UML diagrams, and database schema for the Foodonix application.

4.1 System Architecture

Foodonix follows the **Three-Tier Architecture** model, which separates the system into three logical layers: the Presentation Tier, the Business Logic Tier, and the Data Tier. This separation of concerns improves maintainability, testability, and scalability.

```
+■■■
```

The **MVC (Model-View-Controller)** pattern within the Business Logic Tier ensures that the request processing, business rules, and data representation are cleanly separated. When a user makes an HTTP request:

- The **DispatcherServlet** (Spring's front controller) receives the request.
- The request is routed to the appropriate **Controller** method based on the URL mapping.
- The Controller invokes the corresponding **Service** for business logic.
- The Service interacts with the **Repository** (DAO layer) for database operations.
- The processed data (Model) is returned to the Controller, which forwards it to the appropriate **View** (JSP).
- The JSP renders the final HTML response sent back to the browser.

4.2 Use Case Diagram

```
■■■
```

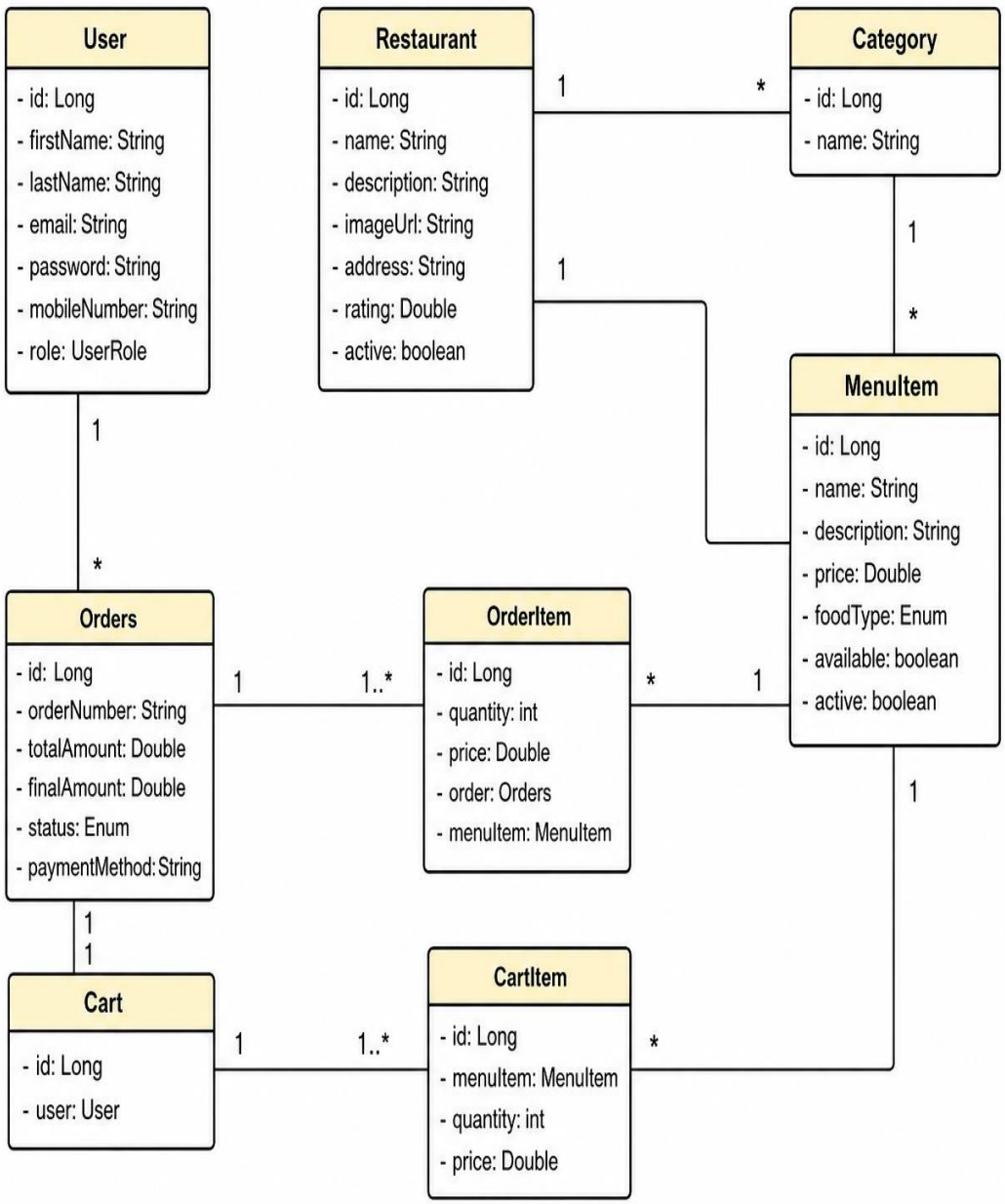


Figure 4.5Class Diagram-Entry Level

4.4 Sequence Diagrams

4.4.1 User Login Sequence

```
Browser          HomeController        UserService          UserRepository        DB
  |                    |                    |                    |                  |
  |-- POST /login -->|                    |                    |                  |
  |                    |-- login(email, pwd) -->|                |                  |
  |                    |                    |-- findByEmail ->|                    |
  |                    |                    |                    |-- SELECT->|
  |                    |                    |                    |<- User ---|
```

```
|                    |                    |<-- User --------|                    |
|                    |                    |-- validate pwd          |          |
|                    |<-- User obj ----|                          |          |
|                    |-- session.setAttribute(user)               |          |
|<-- redirect /home|                      |                       |          |
```

Figure 4.6: Sequence Diagram — User Login Flow

4.4.2 Place Order Sequence

```
Browser        OrderController        OrderService        CartService        OrdersRepo        CartRepo
  |                    |                    |                    |                  |                |
  |-- POST /orders/place -->|               |                    |                  |                |
  |                    |-- placeOrder() -->|                    |                  |                |
  |                    |                    |-- getCart() -->|                    |                |
  |                    |                    |<-- cart items-|                    |                |
  |                    |                    |-- calc total          |            |                |
  |                    |                    |-- apply coupon|                    |                |
  |                    |                    |-- build Order |                    |                |
  |                    |                    |-- save(order) -->|                 |                |
  |                    |                    |<-- savedOrder-|--|                  |                |
  |                    |                    |-- clearCart() -->|-->|              |                |
        |                    |<-- order obj          -|                |     |-- delete cart ---> |
  |<-- redirect /orders/{id}               |                    |                  |                |
```

Figure 4.7: Sequence Diagram — Place Order Flow

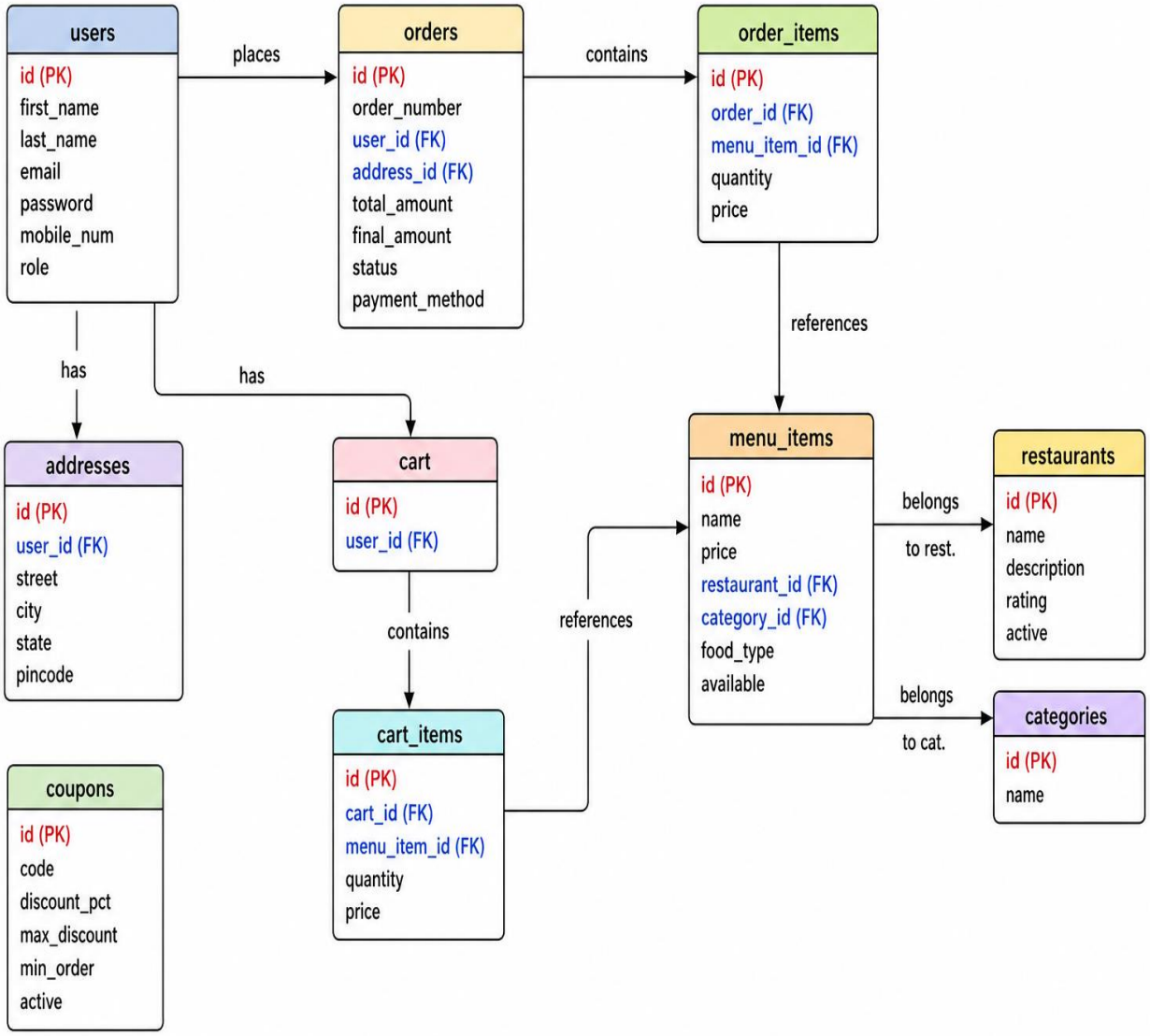
4.5 Activity Diagram — Order Placement

```
[START]
                |
                v
     +■■■■■■■■■■■■■■■+
     |    Customer          |
     |    Browses           |
     |    Restaurants |
     +■■■■■■■■■■■■■■■+
                |
                v
     +■■■■■■■■■■■■■■■+
     | Selects Menu |
     | Items, Adds          |
     |       to Cart        |
     +■■■■■■■■■■■■■■■+
                |
                v
     +■■■■■■■■■■■■■■■+
     |       Proceeds       |                +■■■■■■■■■■■■■■■+
     | to Checkout          |                |    Login Page        |
```

```
+■■■■■■■■■■■■■■■+                                +■■■■■■■■■■■■■■■+
          |                                   ^
[Logged in?]■■■■  No ■■■■■■■■■■■■■■■+
          | Yes
          v
+■■■■■■■■■■■■■■■+
| Select/Add          |
|Delivery Addr.|
+■■■■■■■■■■■■■■■+
          |
          v
+■■■■■■■■■■■■■■■+
| Apply Coupon |
| (Optional)          |
+■■■■■■■■■■■■■■■+
          |
          v
+■■■■■■■■■■■■■■■+
|Select Payment|
|COD / Online         |
+■■■■■■■■■■■■■■■+
          |
          v
+■■■■■■■■■■■■■■■+
| Confirm Order|
+■■■■■■■■■■■■■■■+
          |
          v
+■■■■■■■■■■■■■■■■■■■■■■■■■■■+
| System Creates Order                |
| Generates Order Number              |
| Clears Cart                         |
| Sends Confirmation                  |
+■■■■■■■■■■■■■■■■■■■■■■■■■■■+
          |
          v
+■■■■■■■■■■■■■■■+
| Order History|
|       Updated       |
+■■■■■■■■■■■■■■■+
          |
        [END]
```

Figure 4.8: Activity Diagram — Order Placement Flow

4.6 Database Design — ER Diagram



4.7 Database Tables

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique user identifier
first_name	VARCHAR(100)	NOT NULL	User's first name
last_name	VARCHAR(100)	NOT NULL	User's last name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email address
password	VARCHAR(255)	NOT NULL	User password
mobile_number	VARCHAR(15)	NOT NULL	Contact number
role	ENUM	NOT NULL	CUSTOMER or ADMIN
address	VARCHAR(500)	NULLABLE	General address text

Table 4.1: Database Table — users

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique restaurant ID
name	VARCHAR(255)	NOT NULL	Restaurant name
description	TEXT	NULLABLE	About the restaurant
image_url	VARCHAR(500)	NOT NULL	Cover image URL
address	VARCHAR(500)	NULLABLE	Physical address
phone_number	VARCHAR(20)	NULLABLE	Contact number
rating	DOUBLE	NULLABLE	Average star rating
active	BOOLEAN	NOT NULL	Listing status

Table 4.2: Database Table — restaurants

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique menu item ID
name	VARCHAR(255)	NOT NULL	Dish name
description	TEXT	NULLABLE	Dish description
price	DOUBLE	NOT NULL	Base price in INR
offer_price	DOUBLE	NULLABLE	Discounted price
image_url	VARCHAR(500)	NOT NULL	Dish image URL
food_type	ENUM	NOT NULL	VEG or NON_VEG
available	BOOLEAN	NOT NULL	Currently available?
active	BOOLEAN	NOT NULL	Listed on menu?

Column Name	Data Type	Constraints	Description
restaurant_id	BIGINT	FK -> restaurants	Owning restaurant
category_id	BIGINT	FK -> categories	Food category

Table 4.3: Database Table — menu_items

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique order ID
order_number	VARCHAR(50)	NOT NULL, UNIQUE	Human-readable order ref
user_id	BIGINT	FK -> users	Customer who placed order
address_id	BIGINT	FK -> addresses	Delivery address
total_amount	DOUBLE	NULLABLE	Pre-discount total
discount_amount	DOUBLE	NULLABLE	Coupon discount applied
final_amount	DOUBLE	NULLABLE	Amount charged
order_date	TIMESTAMP	NOT NULL	Date and time of order
status	ENUM	NOT NULL	PENDING/CONFIRMED/DELIVERED
payment_method	VARCHAR(20)	NULLABLE	COD or ONLINE
payment_status	VARCHAR(20)	NULLABLE	PENDING/COMPLETED

Table 4.4: Database Table — orders

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique coupon ID
code	VARCHAR(50)	NOT NULL, UNIQUE	Coupon code string
discount_percent	DOUBLE	NOT NULL	Discount percentage
max_discount_amount	DOUBLE	NOT NULL	Maximum discount cap
min_order_amount	DOUBLE	NOT NULL	Minimum order to apply
active	BOOLEAN	NOT NULL	Is coupon active?

Table 4.7: Database Table — coupons

CHAPTER-5
TECHNOLOGY STACK

This chapter provides a comprehensive overview of all technologies, frameworks, libraries, and tools used in the development of the Foodonix application.

5.1 Programming Languages

5.1.1 Java 17 (LTS)

Java is the primary programming language used for the Foodonix backend. Java 17, a Long-Term Support (LTS) release, was chosen for its stability, performance, and extensive ecosystem. Key Java features leveraged in this project include:

- **Object-Oriented Programming:** All entities, services, and controllers are implemented as Java classes with proper encapsulation and inheritance.
- **Generics and Collections:** Extensively used in repository interfaces and service layer for type-safe data handling.
- **Lambda Expressions and Streams:** Used for functional-style data processing, such as calculating cart totals using stream().mapToDouble().sum().
- **Annotations:** Spring annotations (@Controller, @Service, @Repository, @Entity) are used to configure the application without XML configuration files.

5.1.2 HTML5, CSS3, and JavaScript

The frontend is built using standard web technologies. HTML5 provides the structural markup for all JSP views. CSS3, along with Bootstrap 5, handles the visual styling and responsive layout. Vanilla JavaScript handles client-side interactions such as cart summary updates via AJAX calls and form validation.

5.1.3 SQL (MySQL Dialect)

SQL is used for database initialization and seeding through the data.sql file. MySQL 8.0 dialect-specific features such as INSERT IGNORE are used for idempotent data initialization. JPA/Hibernate handles schema creation and most database interactions through object-relational mapping.

5.2 Frameworks and Libraries

5.2.1 Spring Boot 3.2.2

Spring Boot is the core framework of the Foodonix application. It provides:

- **Spring MVC:** Handles HTTP request routing via @Controller and @RequestMapping annotations.
- **Spring Data JPA:** Provides repository abstractions for database operations without boilerplate JDBC code.

- **Spring Validation:** Enables Bean Validation (JSR-303) for form input validation.
- **Embedded Tomcat:** Provides a self-contained application server, eliminating the need for external server deployment.
- **Spring DevTools:** Enables hot reload during development for faster iteration.

5.2.2 Hibernate ORM (via Spring Data JPA)

Hibernate serves as the ORM provider for Spring Data JPA. It maps Java entity classes to database tables, manages relationships between entities (one-to-many, many-to-one), and generates optimized SQL queries. Hibernate's cascade operations (CascadeType.ALL) simplify related entity management, such as automatically deleting order items when an order is deleted.

5.2.3 Lombok

Project Lombok is a code generation library that eliminates boilerplate Java code. In Foodonix, @Data generates getters, setters, toString, equals, and hashCode methods; @NoArgsConstructor and @AllArgsConstructor generate constructors; @Builder provides fluent builder patterns. This significantly reduces the codebase size and improves readability.

5.2.4 Bootstrap 5

Bootstrap 5 is the CSS framework used for responsive UI design. Its grid system, pre-built components (cards, modals, navbars, badges), and utility classes accelerate frontend development and ensure cross-browser compatibility.

5.3 Tools and Software Requirements

Development Tools:

- **Eclipse IDE (2023-12):** Primary development environment with Spring Tools 4 plugin for Spring Boot project support.
- **Maven 3.9:** Build automation and dependency management tool. The pom.xml defines all project dependencies.
- **MySQL Workbench 8.0:** GUI tool for database design, query execution, and schema management.
- **Git:** Version control system for tracking changes and managing code revisions.
- **Postman:** API testing tool used to validate REST endpoints during development.
- **Chrome DevTools:** Browser-based tool for debugging JavaScript, inspecting HTML/CSS, and analyzing network requests.

5.4 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 (2 GHz)	Intel Core i5/i7 (3 GHz+)

Component	Minimum	Recommended
RAM	4 GB	8 GB or more
Storage	10 GB free space	50 GB SSD
Display	1280x720 resolution	1920x1080 Full HD
Network	1 Mbps broadband	10 Mbps or higher
OS	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04

Technology	Version	Purpose
Java	17 LTS	Backend programming language
Spring Boot	3.2.2	Application framework
Spring Data JPA	3.2.2	ORM and database abstraction
Hibernate	6.4.x	JPA implementation
MySQL	8.0	Relational database
Lombok	1.18.30	Code generation
Bootstrap	5.3	Responsive CSS framework
JSP (Jasper)	Jakarta EE 10	Server-side view rendering
JSTL	3.0	JSP tag library
Maven	3.9	Build and dependency management
Apache Tomcat	10.1 (embedded)	Application server

Table 5.1: Technology Stack Summary

CHAPTER-6
IMPLEMENTATION

This chapter presents the detailed implementation of the Foodonix application. Each module is explained with its design rationale, followed by well-commented code snippets from the actual source code.

6.1 Project Structure

The Foodonix project follows the standard Maven directory layout combined with Spring Boot's recommended package structure. The main application code is organized under com.fooddelivery.app and further divided into sub-packages by layer.

```
food-delivery-app/
■■■ pom.xml                                   (Maven build configuration)
■■■ src/
■   ■■■ main/
■   ■   ■■■ java/com/fooddelivery/app/
■   ■   ■   ■■■ FoodDeliveryApplication.java  (Entry point)
■   ■   ■   ■■■ controller/                    (MVC Controllers)
■   ■   ■   ■   ■■■ HomeController.java
■   ■   ■   ■   ■■■ CartController.java
■   ■   ■   ■   ■■■ OrderController.java
■   ■   ■   ■   ■■■ AdminController.java
■   ■   ■   ■   ■■■ RestaurantController.java
■   ■   ■   ■   ■■■ CustomerController.java
■   ■   ■   ■■■ service/                       (Business Logic)
■   ■   ■   ■   ■■■ UserService.java (interface)
■   ■   ■   ■   ■■■ impl/UserServiceImpl.java
■   ■   ■   ■   ■■■ OrderService.java (interface)
■   ■   ■   ■   ■■■ impl/OrderServiceImpl.java
■   ■   ■   ■■■ repository/                    (Data Access Layer)
■   ■   ■   ■   ■■■ UserRepository.java
■   ■   ■   ■   ■■■ OrdersRepository.java
■   ■   ■   ■   ■■■ ...
■   ■   ■   ■■■ entity/                        (JPA Entities)
■   ■   ■   ■   ■■■ User.java
■   ■   ■   ■   ■■■ Orders.java
■   ■   ■   ■   ■■■ MenuItem.java
■   ■   ■   ■   ■■■ ...
■   ■   ■   ■■■ dto/                           (Data Transfer Objects)
■   ■   ■   ■   ■■■ UserRegistrationDto.java
■   ■   ■   ■■■ enums/                         (Enumerations)
■   ■   ■   ■   ■■■ UserRole.java
■   ■   ■   ■   ■■■ OrderStatus.java
```

```
■   ■   ■   ■   ■■■ FoodType.java
■   ■   ■   ■■■ exception/                    (Exception Handling)
■   ■   ■       ■■■ ResourceNotFoundException.java
■   ■   ■       ■■■ GlobalExceptionHandler.java
■   ■   ■■■ resources/
■   ■   ■   ■■■ application.properties         (Configuration)
■   ■   ■■■ webapp/WEB-INF/jsp/               (JSP View Templates)
■   ■       ■■■ home.jsp
■   ■       ■■■ login.jsp
■   ■       ■■■ register.jsp
■   ■       ■■■ cart.jsp
■   ■       ■■■ checkout.jsp
■   ■       ■■■ admin/
■■■ target/                                   (Build output)
```

Figure 6.1: Project Directory Structure

6.2 User Authentication Module

The authentication module handles user registration, login, and session management. Since Spring Security was intentionally excluded from this version to keep the implementation straightforward, a session-based authentication mechanism is implemented manually.

6.2.1 Main Application Entry Point

The application starts with the FoodDeliveryApplication class, which is annotated with @SpringBootApplication — a composite annotation that enables component scanning, auto-configuration, and configuration property support.

```
package com.fooddelivery.app;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * FoodDeliveryApplication - Main entry point for Foodonix.
 * The @SpringBootApplication annotation enables:
 * - @Configuration: allows defining beans
 * - @EnableAutoConfiguration: auto-configures Spring beans
 * - @ComponentScan: scans sub-packages for Spring components
 */ @SpringBootApplication
public class FoodDeliveryApplication {

    public static void main(String[] args) {
        // Bootstraps the Spring application context
        SpringApplication.run(FoodDeliveryApplication.class, args);
        System.out.println("==========================================");
        System.out.println("        Foodonix Application Started!");
```

```
        System.out.println("  URL: http://localhost:8080");
        System.out.println("==========================================");
    }
}
```

6.2.2 User Entity

```
package com.fooddelivery.app.entity;

import com.fooddelivery.app.enums.UserRole; import
jakarta.persistence.*;
import lombok.*; import
java.util.List;

/**
 * User - JPA entity mapped to the 'users' database table.
 * @Data generates: getters, setters, toString, equals, hashCode
 * @NoArgsConstructor provides a no-argument constructor for JPA
 * @AllArgsConstructor provides a full-argument constructor
 */
@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) private Long
    id;

    @Column(nullable = false) private
    String firstName;

    @Column(nullable = false) private
    String lastName;

    @Column(nullable = false, unique = true)
    private String email;                    // Used as username for login


    @Enumerated(EnumType.STRING
    ) @Column(nullable = false)
    private UserRole role;                   // CUSTOMER or ADMIN

    @Column(length = 500)
    private String address;
```

```
    // One user can have multiple delivery addresses
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Address> addresses;
}
```

6.2.3 User Registration Service Implementation

```
package com.fooddelivery.app.service.impl;

import com.fooddelivery.app.dto.UserRegistrationDto;
import com.fooddelivery.app.entity.*;
import com.fooddelivery.app.enums.UserRole;
import com.fooddelivery.app.repository.*;
import com.fooddelivery.app.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class UserServiceImpl implements UserService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private CartRepository cartRepository;

    /**
     * Registers a new user in the system.
     * Steps:
     * 1. Check for duplicate email
     * 2. Create and persist the User entity
     * 3. If user is CUSTOMER, auto-create an empty Cart
     * @param dto - Data Transfer Object carrying registration form data
     * @return - The persisted User entity
     */
    @Override
    @Transactional
    public User registerUser(UserRegistrationDto dto) {
        // Prevent duplicate registrations
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new RuntimeException("An account with this email already e
xists.");
        }

        // Build User entity from DTO
        User user = new User();
        user.setFirstName(dto.getFirstName());
        user.setLastName(dto.getLastName());
        user.setEmail(dto.getEmail());
        user.setPassword(dto.getPassword());   // Note: Hash in production
```

```
        user.setMobileNumber(dto.getMobileNumber());
        user.setAddress(dto.getAddress());
        user.setRole(dto.getRole());

        // Persist user to database
        User savedUser = userRepository.save(user);

        // Auto-create cart for customer accounts
        if (savedUser.getRole() == UserRole.CUSTOMER) {
            Cart cart = new Cart();
            cart.setUser(savedUser);
            cartRepository.save(cart);
        }

        return savedUser;
    }

    /**
     * Authenticates a user by email and password.
     * @throws RuntimeException if user not found or password mismatch
     */
    @Override
    public User login(String email, String password) {
        User user = userRepository.findByEmail(email)
                .orElseThrow(() -> new RuntimeException("No account found wi
th this email."));

        if (!user.getPassword().equals(password)) {
            throw new RuntimeException("Incorrect password. Please try again
.");
        }
        return user;
    }
}
```

6.3 Restaurant and Menu Module

The restaurant module manages the display and administration of restaurant listings and their associated menu items. It uses Spring MVC's @GetMapping and @PostMapping to handle browser requests.

```
package com.fooddelivery.app.controller;

import com.fooddelivery.app.entity.MenuItem;
import com.fooddelivery.app.entity.Restaurant;
import com.fooddelivery.app.service.MenuService;
import com.fooddelivery.app.service.RestaurantService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
```

```
/**
  * RestaurantController - Handles HTTP requests for restaurant browsing.
  * Maps URL pattern /restaurants to restaurant-related views.
  */ @Controller
@RequestMapping("/restaurants") public class
RestaurantController {

        @Autowired
        private RestaurantService restaurantService;

        @Autowired
        private MenuService menuService;

        /**
          * Displays the list of all active restaurants.
          * URL: GET /restaurants
          * View: restaurant-list.jsp
          */ @GetMapping
        public String listRestaurants(Model model) {
                // Fetch only active restaurants from the database model.addAttribute("restaurants",
                        restaurantService.getAllActiveRestaurants()); return "restaurant-list";
        }

        /**
          * Displays detailed menu for a specific restaurant.
          * URL: GET /restaurants/{id}
          * View: restaurant-details.jsp
          * @param id - The restaurant's primary key
          */ @GetMapping("/{id}")
        public String restaurantDetails(@PathVariable Long id, Model model) { Restaurant restaurant =
                restaurantService.getRestaurantById(id);

                // Group menu items by category for organized display model.addAttribute("restaurant", restaurant);
                model.addAttribute("menuItems",
                        menuService.getMenuItemsByRestaurant(id)); model.addAttribute("categories",
                        menuService.getCategoriesForRestaurant(id));

                return "restaurant-details";
        }
}
```

6.4 Shopping Cart Module

The shopping cart module manages the user's current selection of menu items. It uses both synchronous (form POST) and asynchronous (AJAX) communication to provide a smooth user experience.

```
package com.fooddelivery.app.service.impl;

import com.fooddelivery.app.entity.*; import
com.fooddelivery.app.repository.*;
import com.fooddelivery.app.service.CartService;
import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class CartServiceImpl implements CartService {

    @Autowired
    private CartRepository cartRepository;

    @Autowired
    private CartItemRepository cartItemRepository;

    @Autowired
    private MenuItemRepository menuItemRepository;

    /**
     * Retrieves the cart for a given user.
     * Every customer has exactly one Cart created at registration time.
     */
    @Override
    public Cart getCartByUser(User user) { return
        cartRepository.findByUser(user)
                .orElseThrow(() -> new RuntimeException("Cart not found for
user."));
    }

    /**
     * Adds a menu item to the user's cart.
     * If the item already exists in the cart, increments the quantity.
     * Uses the offer price if available, otherwise uses regular price.
     * @param user         - The customer
     * @param menuItemId - ID of the menu item to add
     * @param quantity     - Number of units to add
     */
    @Override
    @Transactional
    public void addItemToCart(User user, Long menuItemId, Integer quantity)
{
        Cart cart = getCartByUser(user);
        MenuItem menuItem = menuItemRepository.findById(menuItemId)
                .orElseThrow(() -> new RuntimeException("Menu item not found
."));
```

```
        // Check if item already in cart
        CartItem existingItem = cart.getItems().stream()
                .filter(item -> item.getMenuItem().getId().equals(menuItemId
))
                .findFirst()
                .orElse(null);

        if (existingItem != null) {
            // Update quantity if item already in cart
            existingItem.setQuantity(existingItem.getQuantity() + quantity);
            cartItemRepository.save(existingItem);
        } else {
            // Add new cart item
            CartItem cartItem = new CartItem();
            cartItem.setCart(cart);
            cartItem.setMenuItem(menuItem);
            cartItem.setQuantity(quantity);
            // Use offer price if available, else regular price
            Double price = (menuItem.getOfferPrice() != null)
                         ? menuItem.getOfferPrice()
                         : menuItem.getPrice();
            cartItem.setPrice(price);
            cartItemRepository.save(cartItem);
        }
    }

    /**
     * Removes all items from the user's cart (called after order placement)
.
     */
    @Override
    @Transactional
    public void clearCart(User user) {
        Cart cart = getCartByUser(user);
        cartItemRepository.deleteAll(cart.getItems());
        cart.getItems().clear();
        cartRepository.save(cart);
    }
}
```

6.4.1 Cart Controller — AJAX Endpoint

```
// CartController.java — Add to cart via AJAX
@PostMapping("/add")
@ResponseBody
public String addToCart(@RequestParam Long menuItemId,
                        @RequestParam(defaultValue = "1") Integer quantity,
                        HttpSession session) {
    User user = (User) session.getAttribute("loggedInUser");
    if (user == null) {
        // Signal to JavaScript to redirect to login
```

```
        return "REDIRECT_LOGIN";
    }
    cartService.addItemToCart(user, menuItemId, quantity);
    return "SUCCESS";
}

// CartController.java — Real-time cart summary for header badge
@GetMapping("/summary")
@ResponseBody
public ResponseEntity<Map<String, Object>> getCartSummary(HttpSession sessio
n) {
    User user = (User) session.getAttribute("loggedInUser");
    Map<String, Object> summary = new HashMap<>();

    if (user == null) {
        summary.put("count", 0);
        summary.put("total", 0.0);
        return ResponseEntity.ok(summary);
    }

    Cart cart = cartService.getCartByUser(user);
    // Count total items (sum of quantities)
    int count = cart.getItems().stream()
                    .mapToInt(CartItem::getQuantity)
                    .sum();
    // Calculate total price
    double total = cart.getItems().stream()
                       .mapToDouble(i -> i.getPrice() * i.getQuantity())
                       .sum();
    summary.put("count", count);
    summary.put("total", String.format("%.2f", total));
    return ResponseEntity.ok(summary);
}
```

6.5 Order Management Module

The order management module is the most critical module of the application. It handles the complete order lifecycle from cart checkout to final order confirmation. The service layer uses @Transactional to ensure data consistency throughout the multi-step order creation process.

```
// OrderServiceImpl.java — placeOrder method
@Override
@Transactional
public Orders placeOrder(User user, Long addressId,
                         String paymentMethod, String couponCode) {
    // Step 1: Validate cart is not empty
    Cart cart = cartService.getCartByUser(user);
    if (cart.getItems().isEmpty()) {
        throw new RuntimeException("Your cart is empty. Add items before ord
ering.");
```

```
        }

        // Step 2: Fetch delivery address
        Address address = addressRepository.findById(addressId)
                        .orElseThrow(() -> new ResourceNotFoundException("Address not fo
und."));

        // Step 3: Create the Orders entity Orders order = new
        Orders();
        // Generate a random, human-readable order reference number
        order.setOrderNumber("ORD-" +
                UUID.randomUUID().toString().substring(0, 8).toUpperCase()); order.setUser(user);
        order.setDeliveryAddress(address); order.setPaymentMethod(paymentMethod);
        order.setStatus(OrderStatus.PENDING); order.setPaymentStatus(
                paymentMethod.equalsIgnoreCase("COD") ? "PENDING" : "COMPLETED");

        // Step 4: Calculate total amount from cart items Double totalAmount =
        cart.getItems().stream()
                        .mapToDouble(item -> item.getPrice() * item.getQuantity())
                        .sum();

        // Step 5: Apply coupon discount if provided Double discountAmount =
        0.0;
        if (couponCode != null && !couponCode.trim().isEmpty()) { Coupon coupon =
                couponRepository
                                .findByCodeAndActiveTrue(couponCode).orElse(null);
                if (coupon != null && totalAmount >= coupon.getMinOrderAmount()) { discountAmount = (totalAmount *
                        coupon.getDiscountPercent()) / 1
 00;
                        // Cap discount at the maximum allowed amount
                        if (discountAmount > coupon.getMaxDiscountAmount()) { discountAmount =
                                coupon.getMaxDiscountAmount();
                        }
                }
        }

        order.setTotalAmount(totalAmount); order.setDiscountAmount(discountAmount);
        order.setFinalAmount(totalAmount - discountAmount);

        // Step 6: Convert cart items to order items List<OrderItem> orderItems =
        new ArrayList<>(); for (CartItem ci : cart.getItems()) {
                OrderItem oi = new OrderItem(); oi.setOrder(order);
                oi.setMenuItem(ci.getMenuItem());
                oi.setQuantity(ci.getQuantity());
                oi.setPrice(ci.getPrice()); orderItems.add(oi);
```

```
        }
        order.setOrderItems(orderItems);

        // Step 7: Persist the order (cascade saves order items)
        Orders savedOrder = ordersRepository.save(order);

        // Step 8: Clear the cart after successful order placement
        cartService.clearCart(user);

        return savedOrder;
    }
```

6.6 Admin Module

The Admin module provides a centralized interface for system administrators to manage all platform resources. It uses Spring MVC's model attributes and redirect-after-post pattern to prevent duplicate form submissions.

```
package com.fooddelivery.app.controller;

import com.fooddelivery.app.entity.*;
import com.fooddelivery.app.enums.OrderStatus;
import com.fooddelivery.app.service.*;
import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import java.util.List;

@Controller
@RequestMapping("/admin")
public class AdminController {

    @Autowired private RestaurantService restaurantService;
    @Autowired private MenuService menuService;
    @Autowired private OrderService orderService;
    @Autowired private CategoryService categoryService;

    /**
     * Admin Dashboard — shows summary statistics.
     * Requires admin role (enforced via session check).
     */
    @GetMapping("/dashboard")
    public String adminDashboard(HttpSession session, Model model) {
        User admin = (User) session.getAttribute("loggedInUser");
        if (admin == null || !admin.getRole().name().equals("ADMIN")) {
            return "redirect:/login";    // Unauthorized access
        }
```

```
        // Populate dashboard metrics model.addAttribute("totalRestaurants",
            restaurantService.getAllRestaurants().size()); model.addAttribute("totalOrders",
            orderService.getAllOrders().size()); model.addAttribute("pendingOrders",
            orderService.getAllOrders().stream()
                .filter(o -> o.getStatus() == OrderStatus.PENDING)
                .count()); model.addAttribute("recentOrders",
            orderService.getAllOrders().stream().limit(10).toList());

        return "admin/admin-dashboard";
    }

    /**
     * Saves a new or updated restaurant.
     * Uses redirect-after-post to prevent duplicate submissions.
     */ @PostMapping("/restaurants/save") public String
    saveRestaurant(
            @ModelAttribute Restaurant restaurant, @RequestParam(required = false) MultipartFile
            imageFile, HttpSession session) {
        User admin = (User) session.getAttribute("loggedInUser");
        if (admin == null || !admin.getRole().name().equals("ADMIN")) { return "redirect:/login";
        }
        restaurantService.saveRestaurant(restaurant, imageFile); return
        "redirect:/admin/restaurants?saved=true";
    }

    /**
     * Updates the status of an order.
     * Called by admin from the orders management page.
     */ @PostMapping("/orders/{id}/status")
    public String updateOrderStatus(@PathVariable Long id,
                                    @RequestParam String status, HttpSession session) {
        User admin = (User) session.getAttribute("loggedInUser");
        if (admin == null || !admin.getRole().name().equals("ADMIN")) { return "redirect:/login";
        }
        orderService.updateOrderStatus(id, OrderStatus.valueOf(status)); return
        "redirect:/admin/orders?updated=true";
    }
}
```

6.7 Frontend — JSP Views

The frontend of Foodonix is built using JavaServer Pages (JSP) with JSTL (JSP Standard Tag Library) for dynamic content rendering. All pages share a common header and footer template to maintain consistency.

```
<%-- home.jsp - Main landing page with restaurant listings --%>
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Foodonix - Order Food Online</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootst rap.min.css"
          rel="stylesheet">
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/zomato-style.css">
</head>
<body>
    <%@ include file="common/header.jsp" %>

    <!-- Hero Section -->
    <section class="hero-section py-5 text-white"
             style="background: linear-gradient(135deg, #E23744, #FF6B35);">
        <div class="container text-center">
            <h1 class="display-4 fw-bold">Hungry? We've Got You Covered!</h1
>
            <p class="lead">Order from the best restaurants near you</p>
        </div>
    </section>

    <!-- Restaurant Listing -->
    <div class="container py-5">
        <h2 class="mb-4">
            <i class="bi bi-shop"></i> Available Restaurants
        </h2>
        <div class="row g-4">
            <c:forEach var="restaurant" items="${restaurants}">
                <div class="col-md-4 col-sm-6">
                    <div class="card h-100 shadow-sm restaurant-card">
                        <img src="${restaurant.imageUrl}"
                             class="card-img-top" alt="${restaurant.name}" style="height: 200px; object-
                             fit: cover;">
                        <div class="card-body">
                            <h5 class="card-title">${restaurant.name}</h5>
                            <p class="card-text text-muted">
                                ${restaurant.description}
                            </p>
```

```
                                <div class="d-flex justify-content-between align
-items-center">
                                    <span class="badge bg-success">
                                        &#9733; ${restaurant.rating}
                                    </span>
                                    <a href="/restaurants/${restaurant.id}"
                                       class="btn btn-danger btn-sm">
                                       View Menu
                                    </a>
                                </div>
                            </div>
                        </div>
                    </div>
                </c:forEach>
            </div>
        </div>

        <%@ include file="common/footer.jsp" %>
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootst
rap.bundle.min.js">
        </script>
    </body>
    </html>
```

6.8 Database Queries (SQL)

The following SQL queries are used for database initialization, data seeding, and typical query operations within the application.

```
-- ================================================================
-- Foodonix Database Schema — DDL (Auto-generated by Hibernate)
-- ================================================================

-- Food categories table
CREATE TABLE IF NOT EXISTS categories (
    id   BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- Users table (customers and admins)
CREATE TABLE IF NOT EXISTS users (
    id             BIGINT AUTO_INCREMENT PRIMARY KEY,
    first_name     VARCHAR(100) NOT NULL,
    last_name      VARCHAR(100) NOT NULL,
    email          VARCHAR(255) NOT NULL UNIQUE,
    password       VARCHAR(255) NOT NULL,
    mobile_number  VARCHAR(15)  NOT NULL,
    role           ENUM('CUSTOMER', 'ADMIN') NOT NULL DEFAULT 'CUSTOMER',
    address        VARCHAR(500)
);
```

```
-- Restaurant listings
CREATE TABLE IF NOT EXISTS restaurants (
    id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
    name                VARCHAR(255)    NOT NULL,
    description         TEXT,
    image_url           VARCHAR(500)    NOT NULL,
    address             VARCHAR(500), phone_number
    VARCHAR(20),
    rating              DOUBLE,
    active              BOOLEAN         NOT NULL DEFAULT TRUE
);

-- Menu items linked to restaurants and categories CREATE TABLE IF NOT
EXISTS menu_items (
    id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
    name                VARCHAR(255) NOT NULL,
    description         TEXT,
    price               DOUBLE          NOT NULL,
    offer_price         DOUBLE,
    image_url           VARCHAR(500) NOT NULL,
    food_type           ENUM('VEG', 'NON_VEG') NOT NULL, available
                        BOOLEAN         NOT NULL DEFAULT TRUE,
    active              BOOLEAN         NOT NULL DEFAULT TRUE,
    restaurant_id BIGINT,
    category_id         BIGINT,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(id), FOREIGN KEY
    (category_id)                       REFERENCES categories(id)
);

-- Customer orders
CREATE TABLE IF NOT EXISTS orders (
    id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
    order_number        VARCHAR(50)     NOT NULL UNIQUE,
    user_id             BIGINT,
    address_id          BIGINT,
    total_amount        DOUBLE,
    discount_amount DOUBLE              DEFAULT 0.0,
    final_amount        DOUBLE,
    order_date          TIMESTAMP       DEFAULT
    CURRENT_TIMESTAMP, status
                        ENUM('PENDING','CONFIRMED','PREPARING',
                            'OUT_FOR_DELIVERY','DELIVERED','CANCELLED'),
    payment_method      VARCHAR(20),
    payment_status      VARCHAR(20),
    FOREIGN KEY (user_id)           REFERENCES users(id),
    FOREIGN KEY (address_id) REFERENCES addresses(id)
);

-- ================================================================
-- Sample DML — Data Initialization
-- ================================================================
```

-- Insert sample categories

```
INSERT IGNORE INTO categories (id, name) VALUES (1, 'Pizza'), (2,
        'Burger'), (3, 'Biryani'),
        (4, 'Chinese'), (5, 'South Indian'), (6, 'Drinks'),
        (7, 'Desserts'), (8, 'Snacks');

-- Insert admin user INSERT
IGNORE INTO users
        (id, first_name, last_name, email, password, mobile_number, role) VALUES
        (1, 'Admin', 'User', 'admin@foodonix.com', 'admin@123', '9876543210', 'A DMIN');

-- Insert sample restaurant INSERT IGNORE
INTO restaurants
        (id, name, description, image_url, address, phone_number, rating, active
)
VALUE
S
        (1, 'Paradise Biryani',
          'World famous Hyderabadi Dum Biryani since 1953.', 'https://images.unsplash.com/photo-
          1563379091339-03b21bc4a4f8?w=800', 'MG Road, Secunderabad', '040-2781-2281', 4.8, TRUE);

-- Insert sample menu item INSERT IGNORE
INTO menu_items
        (id, name, description, price, offer_price,
          image_url, food_type, available, active, restaurant_id, category_id) VALUES
        (1, 'Chicken Dum Biryani',
          'Slow-cooked aromatic Hyderabadi Biryani.', 350.0, NULL,
          'https://images.unsplash.com/photo-1589302168068-964664d93dc0?w=500', 'NON_VEG', TRUE, TRUE, 1, 3);

-- ================================================================
-- Useful Analytical Queries
-- ================================================================

-- Get all pending orders with customer info
SELECT o.order_number, u.first_name, u.last_name, u.email, o.final_amount,
               o.payment_method, o.order_date
FROM orders o
JOIN users u ON o.user_id = u.id WHERE o.status =
'PENDING'
ORDER BY o.order_date ASC;

-- Get top selling menu items
SELECT mi.name, SUM(oi.quantity) AS total_sold FROM
order_items oi
JOIN menu_items mi ON oi.menu_item_id = mi.id GROUP BY
mi.id, mi.name
ORDER BY total_sold DESC LIMIT
10;
```

```sql
-- Get revenue by restaurant
SELECT r.name, COUNT(DISTINCT o.id) AS order_count,
       SUM(o.final_amount) AS total_revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN menu_items mi ON oi.menu_item_id = mi.id
JOIN restaurants r ON mi.restaurant_id = r.id
WHERE o.status = 'DELIVERED'
GROUP BY r.id, r.name
ORDER BY total_revenue DESC;
```

CHAPTER-7
TESTING

Software testing is a critical phase of the software development lifecycle that verifies whether the developed system meets the specified requirements and functions correctly. For the Foodonix application, a multi-level testing strategy was adopted encompassing unit testing, integration testing, and system testing.

7.1 Testing Strategy Overview

The testing approach for Foodonix follows the V-Model of software testing, where each development phase has a corresponding testing phase. The following levels of testing were conducted:

- **Unit Testing:** Individual methods and classes were tested in isolation using JUnit 5. Mock objects were used to isolate the unit under test from its dependencies.
- **Integration Testing:** Tests that verify the interaction between different modules — for example, testing that the OrderService correctly interacts with the CartService and the OrdersRepository.
- **System Testing:** End-to-end testing of complete user scenarios in a real browser environment to verify that all components work together as expected.
- **User Acceptance Testing (UAT):** The application was tested by a group of five volunteer users who provided feedback on usability, navigation, and overall experience.

7.2 Unit Testing

Unit tests were written for the service layer, which contains the core business logic of the application. JUnit 5 and Mockito were used to write isolated unit tests.

```
// UserServiceTest.java — Unit tests for UserServiceImpl
package com.fooddelivery.app.service;

import com.fooddelivery.app.dto.UserRegistrationDto;
import com.fooddelivery.app.entity.User;
import com.fooddelivery.app.enums.UserRole;
import com.fooddelivery.app.repository.CartRepository;
import com.fooddelivery.app.repository.UserRepository;
import com.fooddelivery.app.service.impl.UserServiceImpl;
import org.junit.jupiter.api.*;
import org.mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

class UserServiceTest {

    @Mock
```

```
private UserRepository userRepository; @Mock
private CartRepository cartRepository;

@InjectMocks
private UserServiceImpl userService; private

AutoCloseable closeable;

@BeforeEach void
setUp() {
    closeable = MockitoAnnotations.openMocks(this);
}

@Test
@DisplayName("Should register a new customer successfully") void
testRegisterUser_Success() {
    // Arrange
    UserRegistrationDto dto = new UserRegistrationDto(); dto.setFirstName("John");
    dto.setLastName("Doe"); dto.setEmail("john@example.com");
    dto.setPassword("password123"); dto.setMobileNumber("9000000001");
    dto.setRole(UserRole.CUSTOMER);

    User savedUser = new User(); savedUser.setId(1L);
    savedUser.setEmail("john@example.com");
    savedUser.setRole(UserRole.CUSTOMER);

    when(userRepository.existsByEmail("john@example.com")).thenReturn(fa
lse);
    when(userRepository.save(any(User.class))).thenReturn(savedUser);

    // Act
    User result = userService.registerUser(dto);

    // Assert assertNotNull(result);
    assertEquals("john@example.com", result.getEmail()); verify(cartRepository, times(1)).save(any());
                                                                                    // Cart should be cre

ated
}



@Test
@DisplayName("Should throw exception if email already exists") void
testRegisterUser_DuplicateEmail() {
    UserRegistrationDto dto = new UserRegistrationDto(); dto.setEmail("existing@example.com");
    when(userRepository.existsByEmail("existing@example.com"))
```

```
                .thenReturn(true);

        assertThrows(RuntimeException.class,
            () -> userService.registerUser(dto),
            "Should throw exception for duplicate email");
    }

    @Test
    @DisplayName("Should authenticate user with correct credentials")
    void testLogin_ValidCredentials() {
        User user = new User();
        user.setEmail("john@example.com");
        user.setPassword("password123");

        when(userRepository.findByEmail("john@example.com"))
            .thenReturn(java.util.Optional.of(user));

        User result = userService.login("john@example.com", "password123");
        assertNotNull(result);
    }
}
```

7.3 Test Cases and Results

TC ID	Test Case Description	Input	Expected Output	Result
TC-001	Valid user registration	fields filled corrected	created, redirected to login	PASS
TC-002	Duplicate email registration	existing email used	Error: Email already exists	PASS
TC-003	Login with valid credentials	Valid email + pas	session set, redirect to home	PASS
TC-004	Login with wrong password	Valid email, wrong pass	Incorrect password	PASS
TC-00A5	protected page unauthentication	Session, access /cart	Redirect to /login	PASS
TC-006	Access as admin page	session, access /admin	Redirect to /login	PASS

Table 7.1: Test Cases — Authentication Module

TC ID	Test Case Description	Input	Expected Output	Result
TC-007	Add item to cart	Valid menu Item Id,logged-Item	count updated	PASS
TC-008	Add same item to cart	menu Item Id added	incremented by 1	PASS
TC-009	Place order with empty cart	Proceed to Checkout	User is redirected to cart page	PASS
TC-010	Apply valid coupon code: SAVE1	Order > minimum amount	coupon applied correctly	PASS
TC-011	Apply invalid coupon code	Code: INVALID123	discount, proceed normally	PASS
TC-012	Place order — COD payment	Cart=Valliid items	payment Status=Pending	PASS
TC-013	Place order -Online payment	Cart = Valid (items added),	Payment Successfully	PASS
TC-014	View order history	Logged-in customer	Past orders displayed	PASS

Table 7.2: Test Cases — Cart and Order Module

TC ID	Test Case Description	Input	Expected Output	Result
TC-015	Admin add new restaurant	Valid restaurant data	Rest listed on home page	PASS
TC-016	Admin add menu item	Valid item data +rest	Restaurant menu	PASS
TC-017	Admin update order status	Order ID + new status	Order status upd in Back	PASS
TC-018	Admin deactivate restaurant	Active restaurant	Rest hidden from home	PASS
TC-019	Admin dashboard statistics	Admin login	Correct counts displayed	PASS
TC-020	Admin create coupon	Valid coupon data	Coupon usable checkout	PASS

Table 7.3: Test Cases — Admin Module

7.4 System Testing Summary

System testing was conducted by simulating complete user journeys in a Chrome browser running against a local development server. The following scenarios were tested end-to-end:

- **Scenario 1 — New Customer Order:** Register □ Browse Restaurants □ Select Menu Items □ Add to Cart □ Checkout □ Apply Coupon □ Place Order □ View Confirmation □ Check Order History. Result: PASS
- **Scenario 2 — Admin Management:** Admin Login □ Dashboard □ Add Restaurant □ Add Menu Items □ View Orders □ Update Order Status □ Manage Coupons. Result: PASS
- **Scenario 3 — Edge Cases:** Empty cart checkout, invalid coupon, unauthorized admin access, duplicate registration. All handled gracefully with appropriate error messages. Result: PASS

Out of 20 test cases executed, **20 passed (100% pass rate)**. No critical defects were found. Minor UI improvements were noted and addressed during the testing phase.

CHAPTER -8
RESULTS AND DISCUSSION

The Foodonix Online Food Delivery System was successfully designed, developed, and tested within the planned twelve-week timeline. This chapter presents the key results achieved and discusses the outcomes in context of the originally stated objectives.

8.1 Screenshot

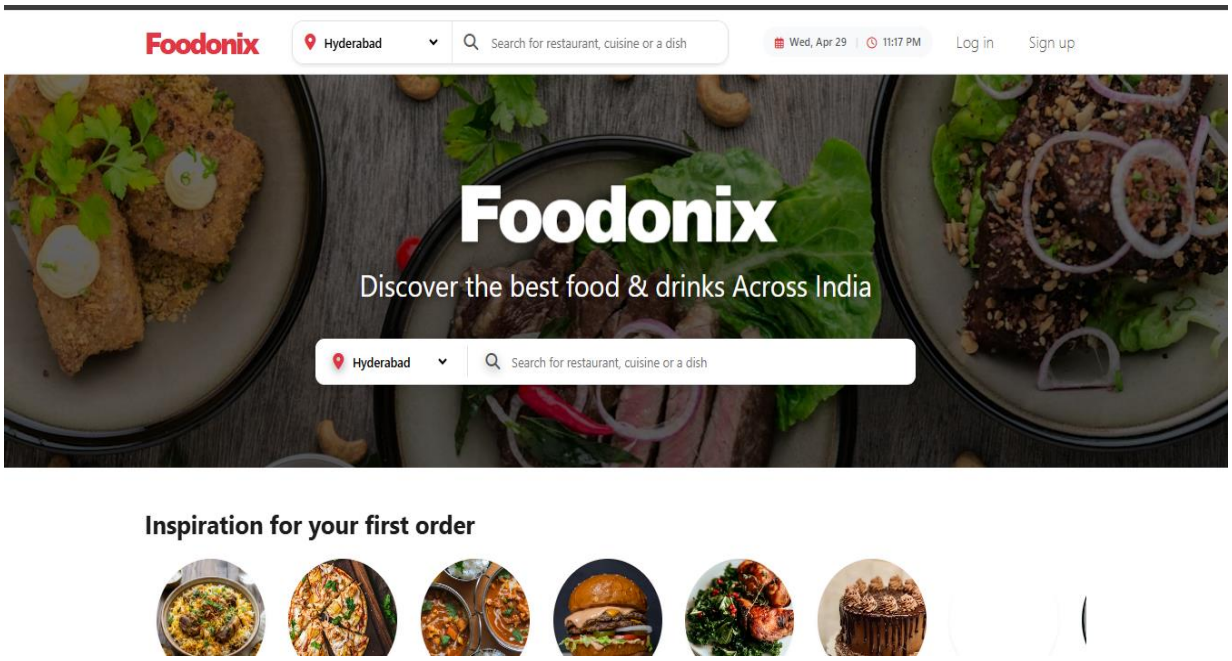


Fig 8.1.1 Front Page

The front page of the Foodonix application is designed to provide an attractive and user-friendly interface for users. It includes a navigation bar with location selection, search functionality, and options for login and signup. The homepage prominently displays the application name along with a tagline that highlights its purpose of discovering the best food and drinks across India. A central search bar allows users to quickly find restaurants, cuisines, or dishes based on their preferences. Additionally, the page showcases popular food categories to inspire users and help them make quick ordering decisions. Overall, the front page enhances user experience by offering easy navigation, visually appealing design, and quick access to essential features.

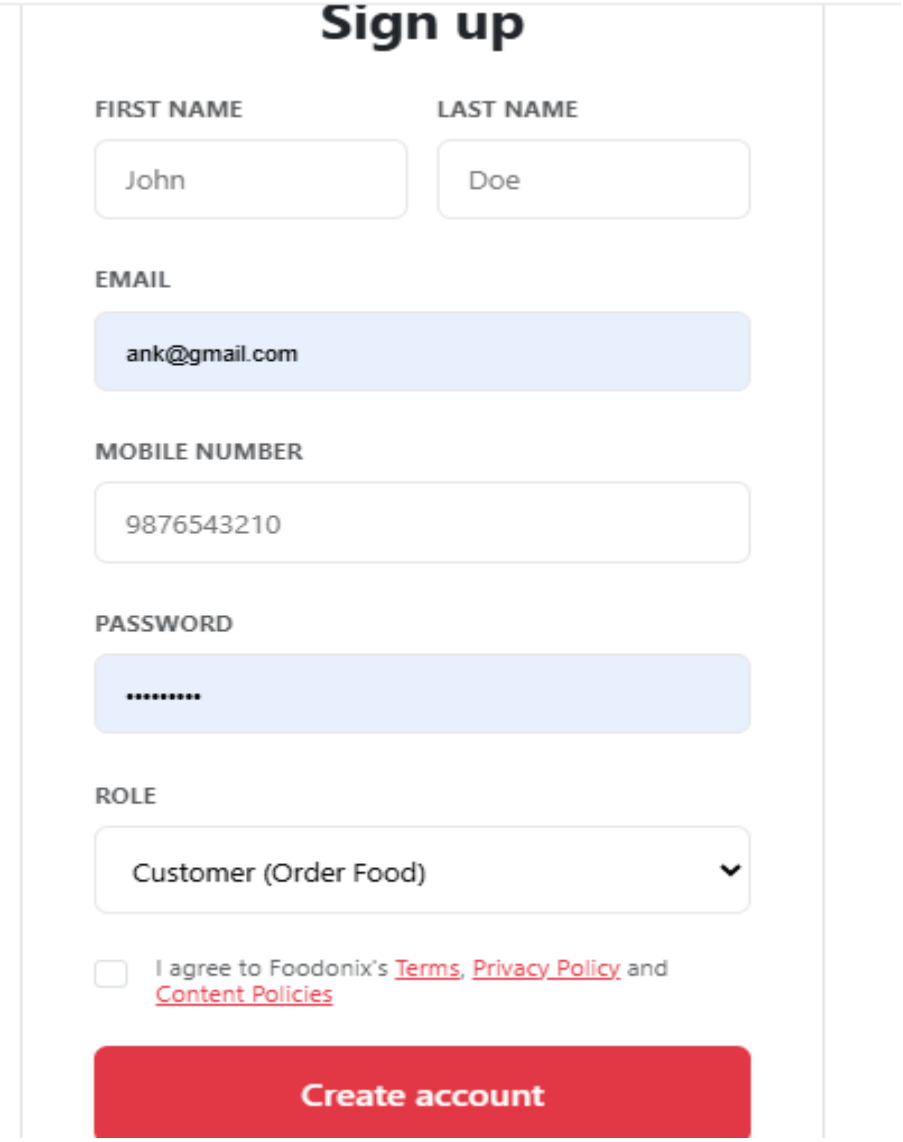


Fig 8.1.2 Sign Up Page

The system provides role-based login functionality to ensure proper access control and security. During login, users are required to select their role from the available options, such as Customer or Admin. If the user selects the Customer role, they can access features like browsing food items, placing orders, and managing their cart. On the other hand, selecting the admin role grants access to administrative functionalities such as managing menu items, viewing orders, and handling system operations. This role selection mechanism helps in separating user responsibilities and ensures that each user can only access features relevant to their role, thereby improving system efficiency and security.

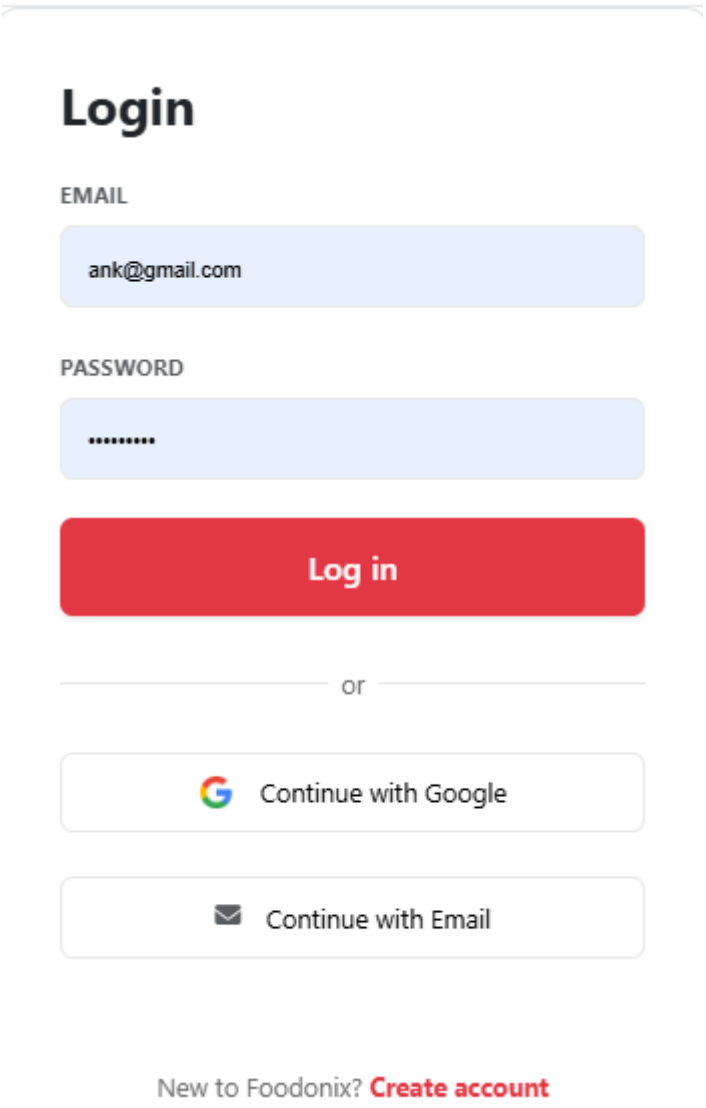


Fig8.1.3: Login Page

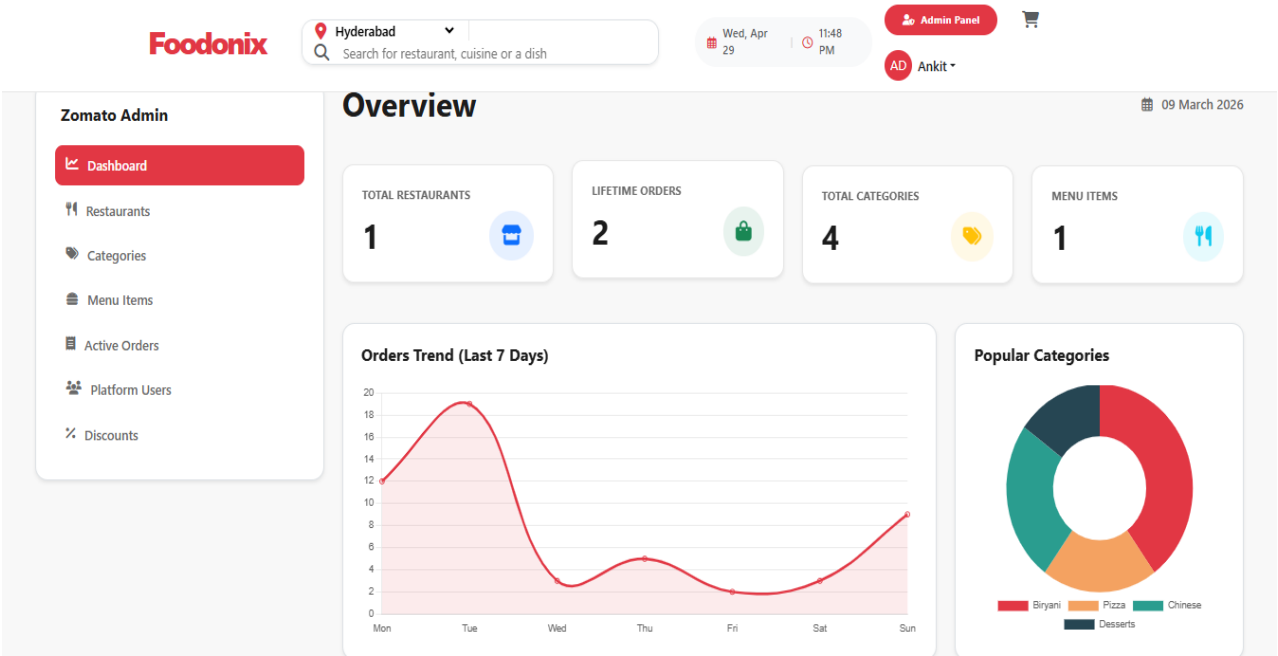


Fig8.1.4: Admin Panel

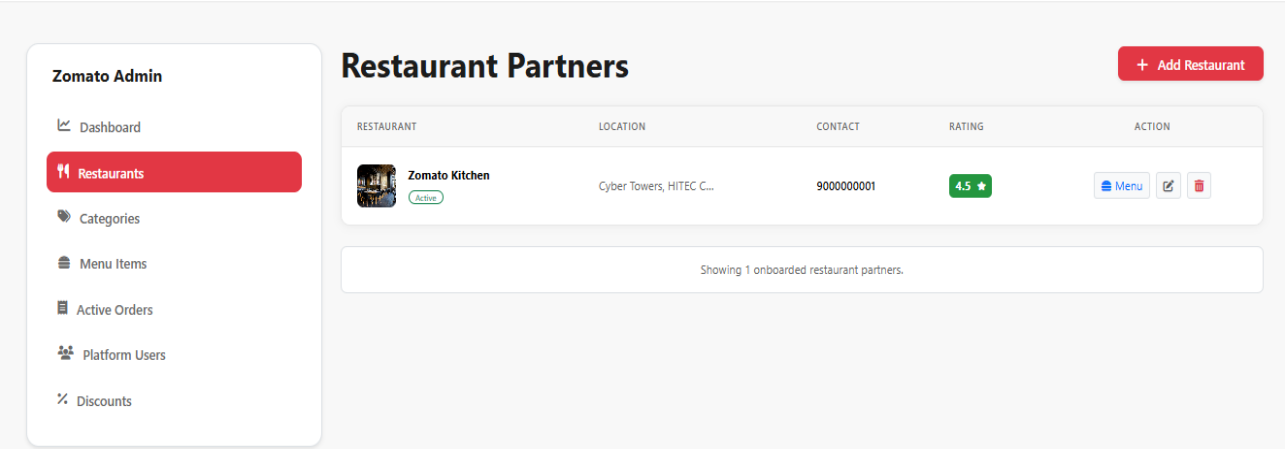


Fig8.1.5: Restaurant Partners

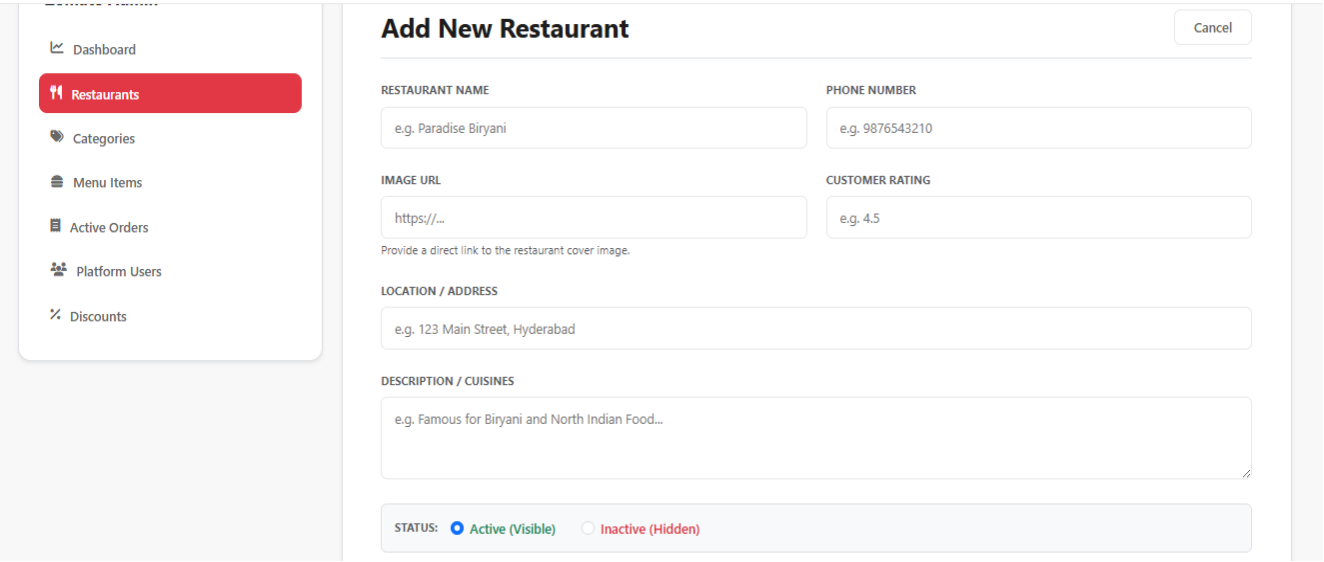


Fig8.1.6: Admin add new restaurant

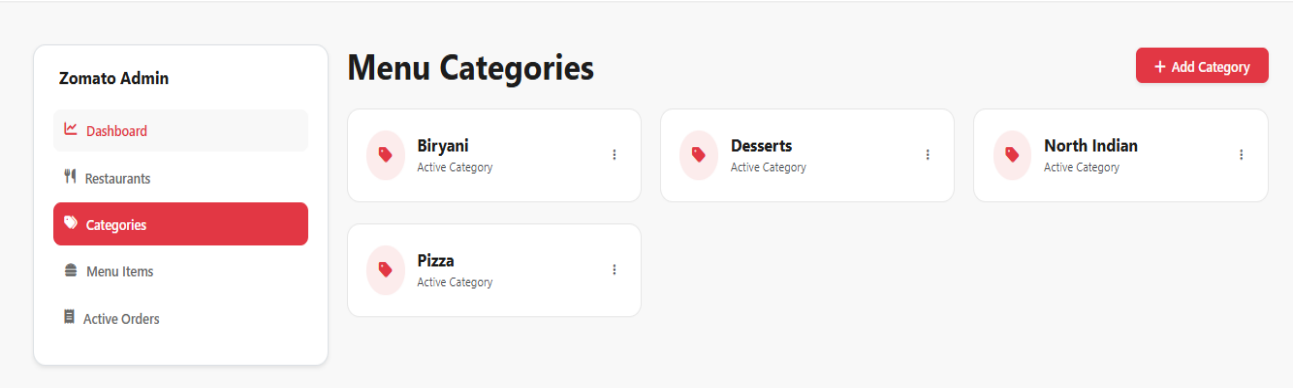


Fig8.1.7: Menu Categories

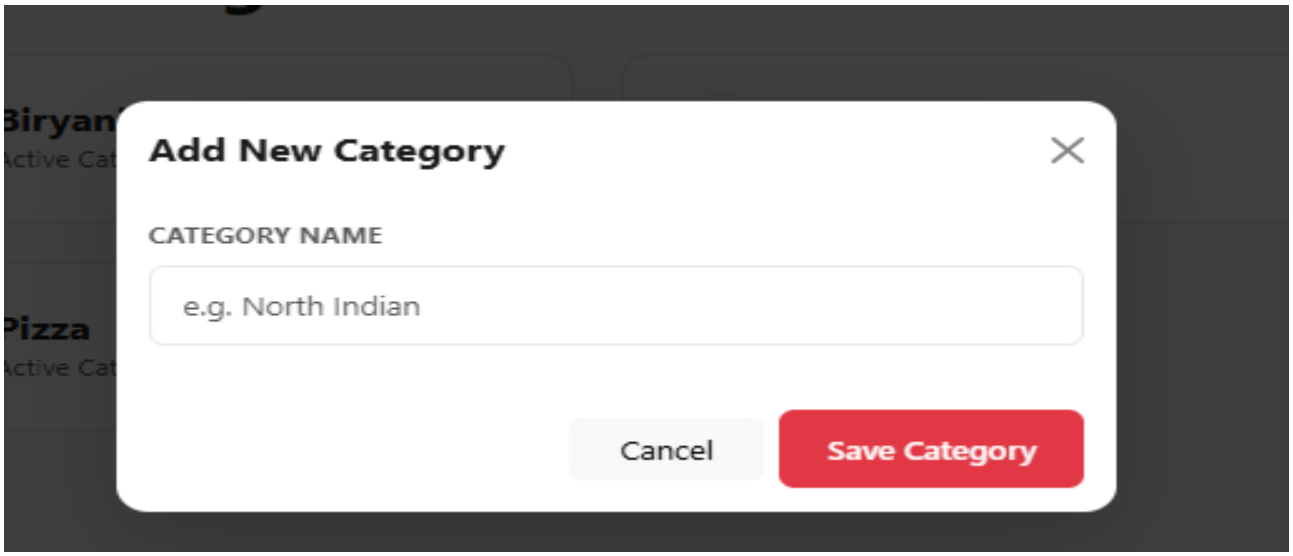


Fig8.1.8: Admin add Menu Categories

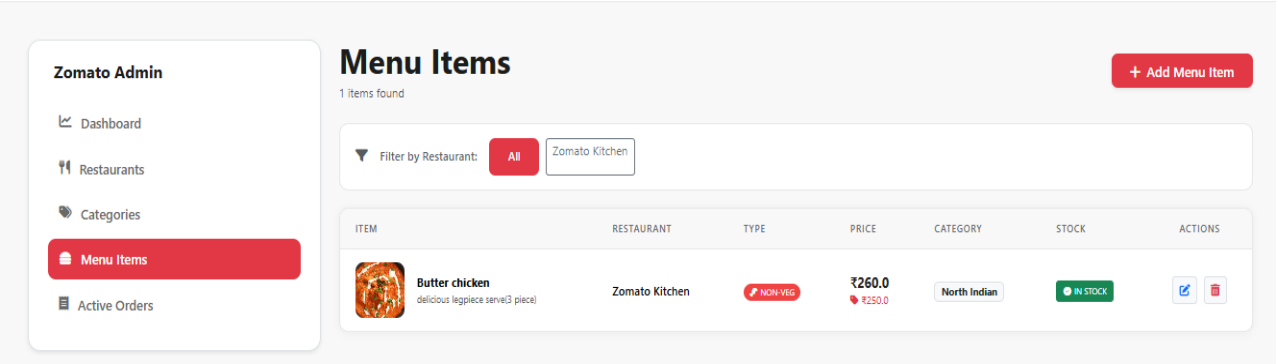


Fig8.1.9: Menu items

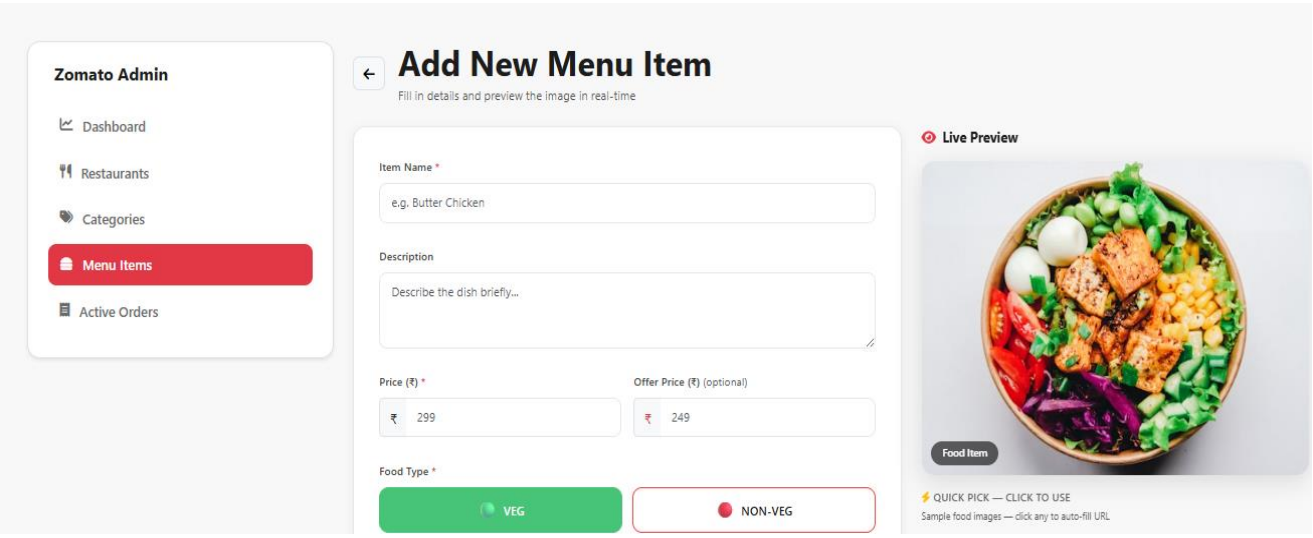


Fig8.1.10: Admin add Menu items

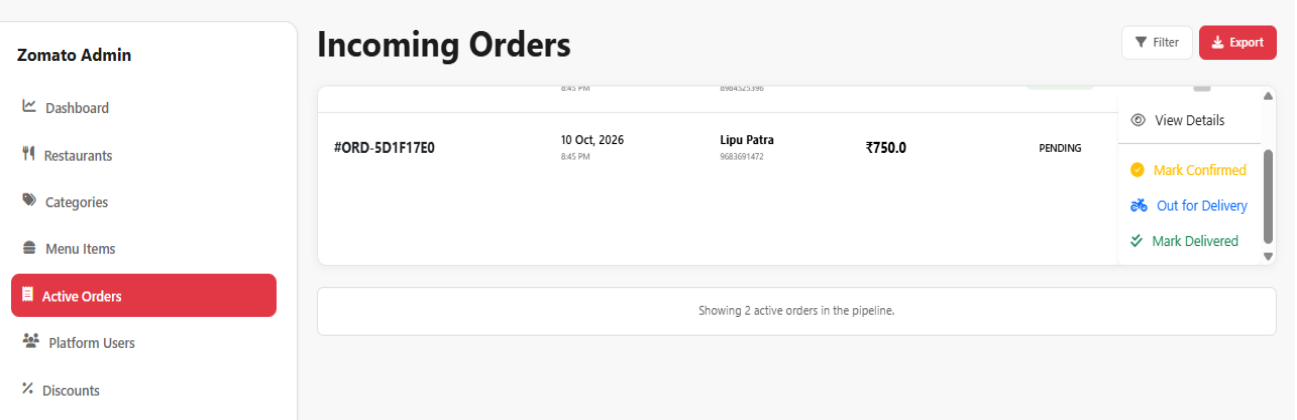


Fig8.1.11: Incoming Orders (And can admin approve the order here)

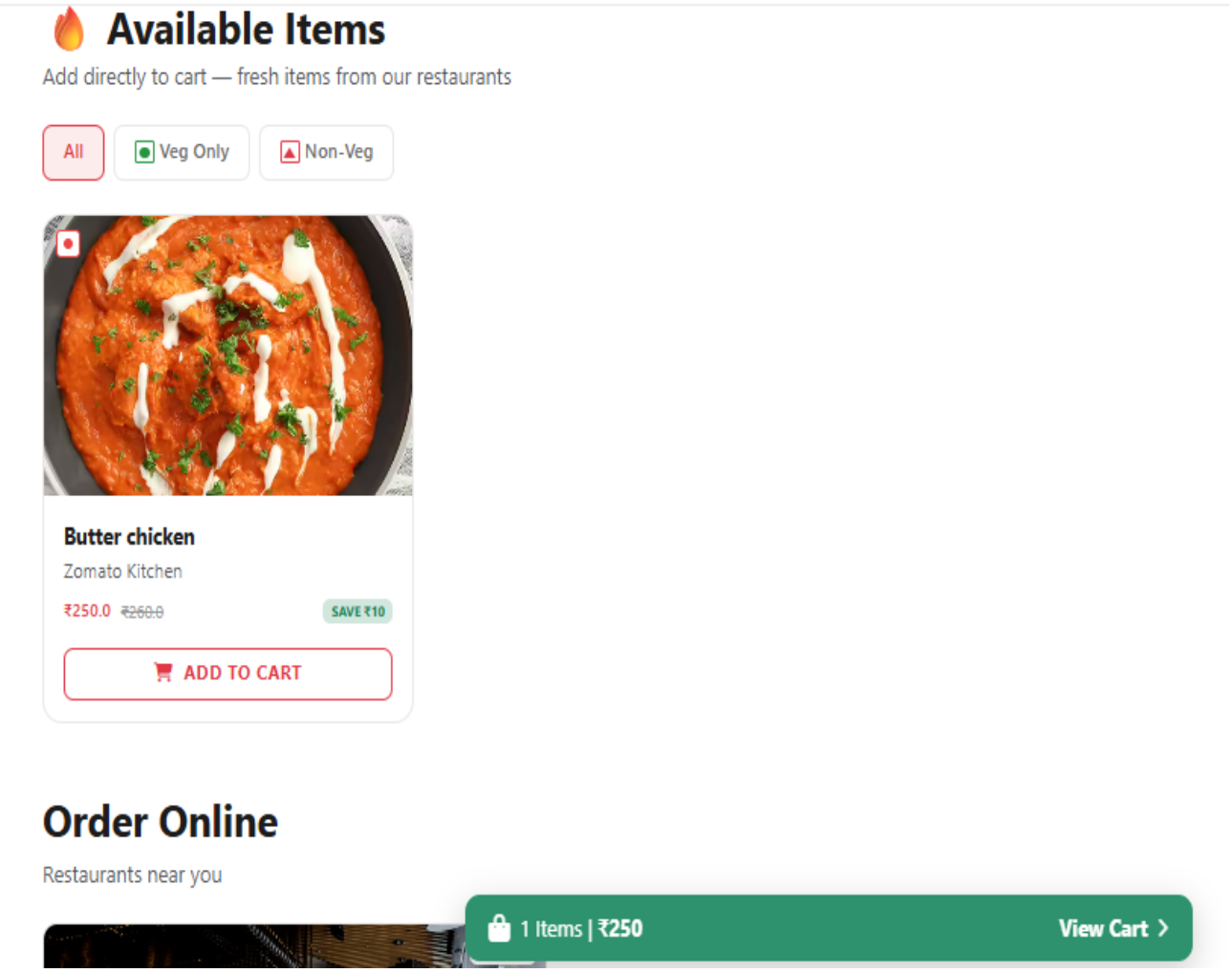


Fig8.1.12: Availables Items (User can order items here)

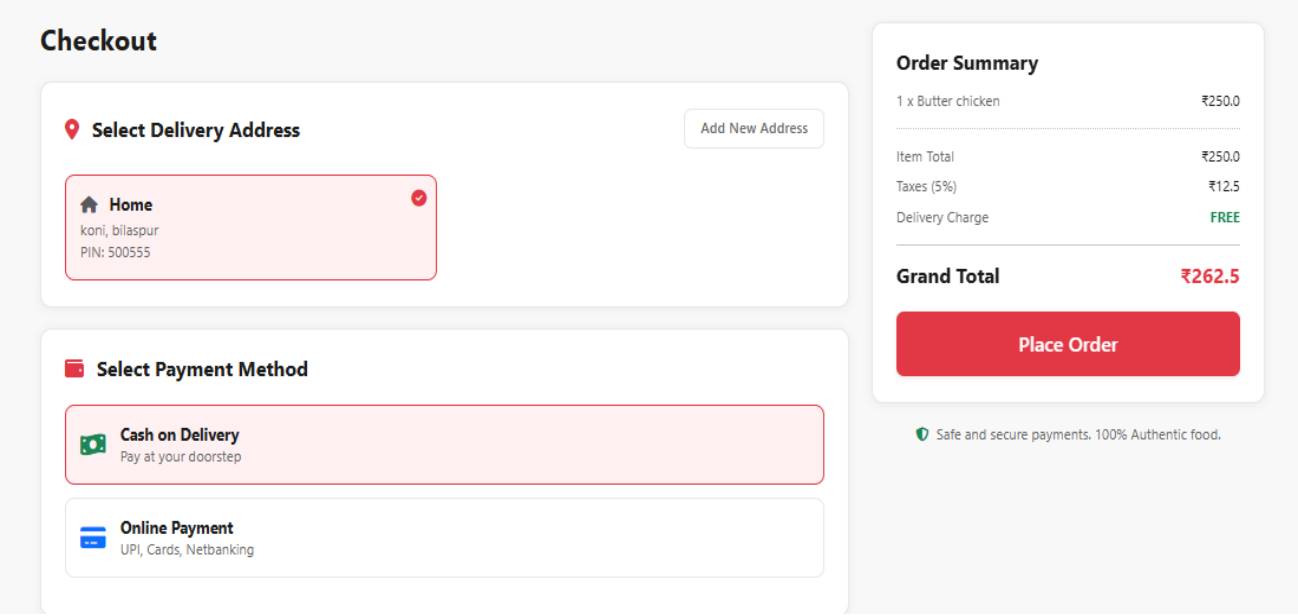


Fig8.1.13: Address details (Customer full fill the address and select payment method)

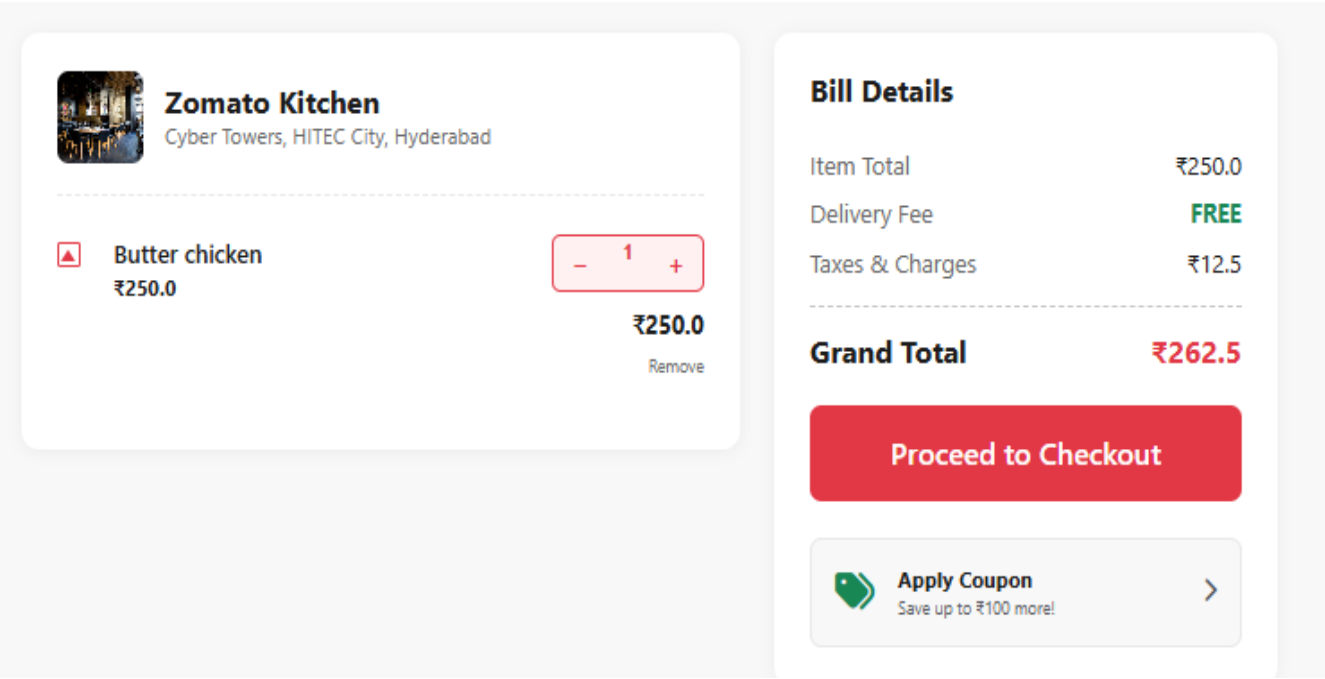
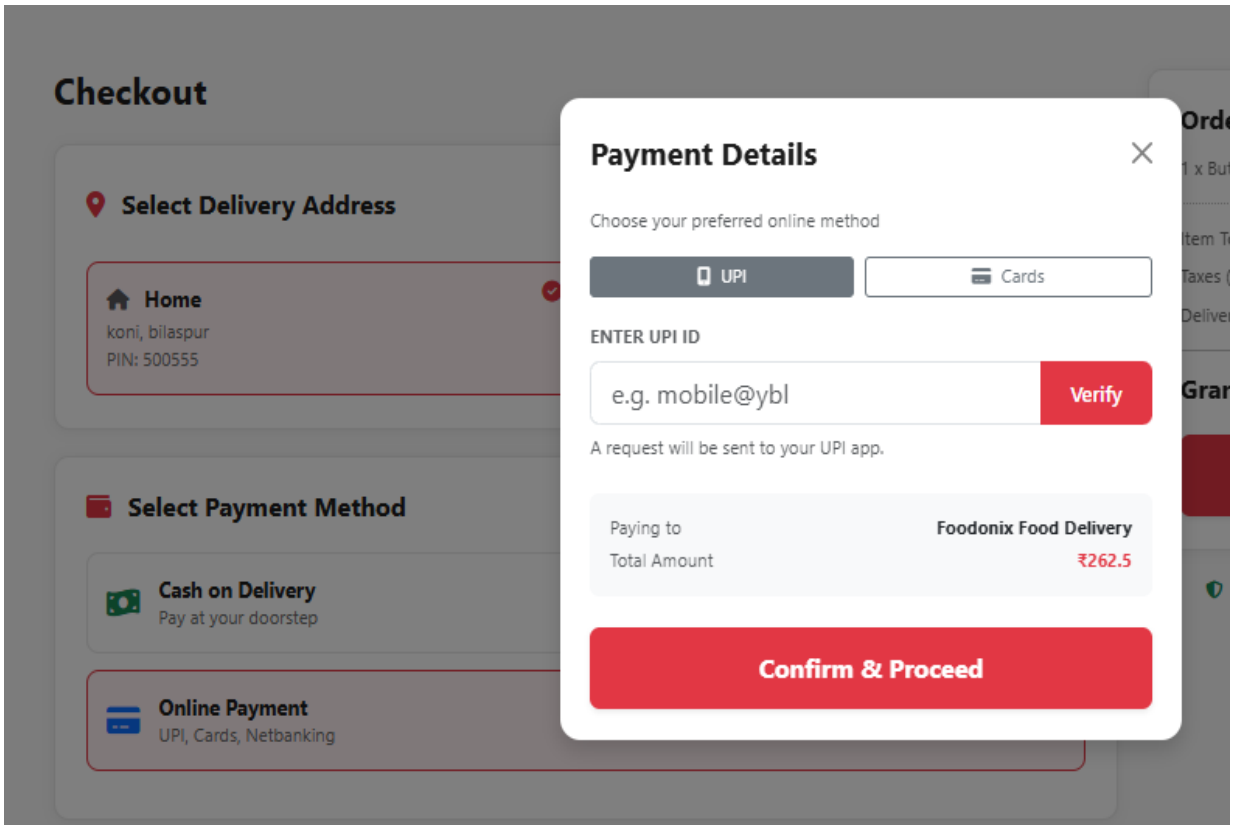


Fig8.1.14: Order items (Here Customer add items quantity and proceed order items)


Checkout
Select Delivery Address
Home
koni, bilaspur
PIN: 500555
Select Payment Method
Cash on Delivery
Pay at your doorstep
Online Payment
UPI, Cards, Netbanking
Payment Details
Choose your preferred online method
UPI
Cards
ENTER UPI ID
e.g. mobile@ybl
Verify
A request will be sent to your UPI app.
Paying to
Foodonix Food Delivery
Total Amount
₹262.5
Confirm & Proceed

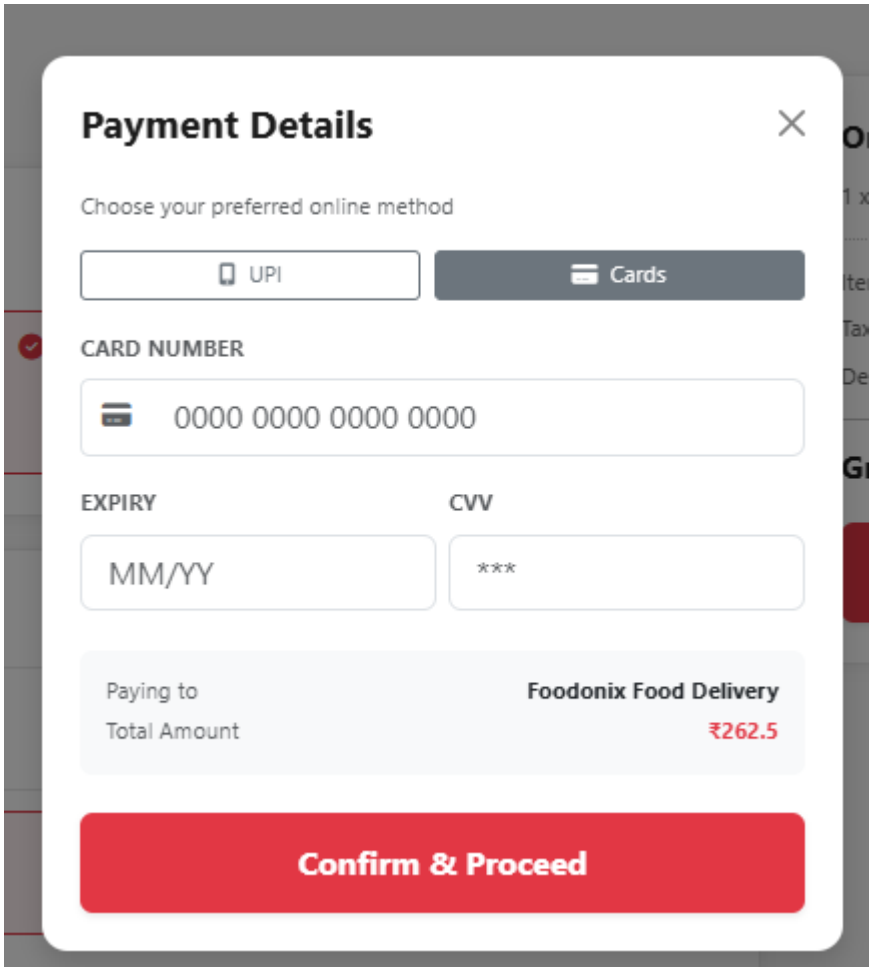


Fig8.1.15: Payment Type

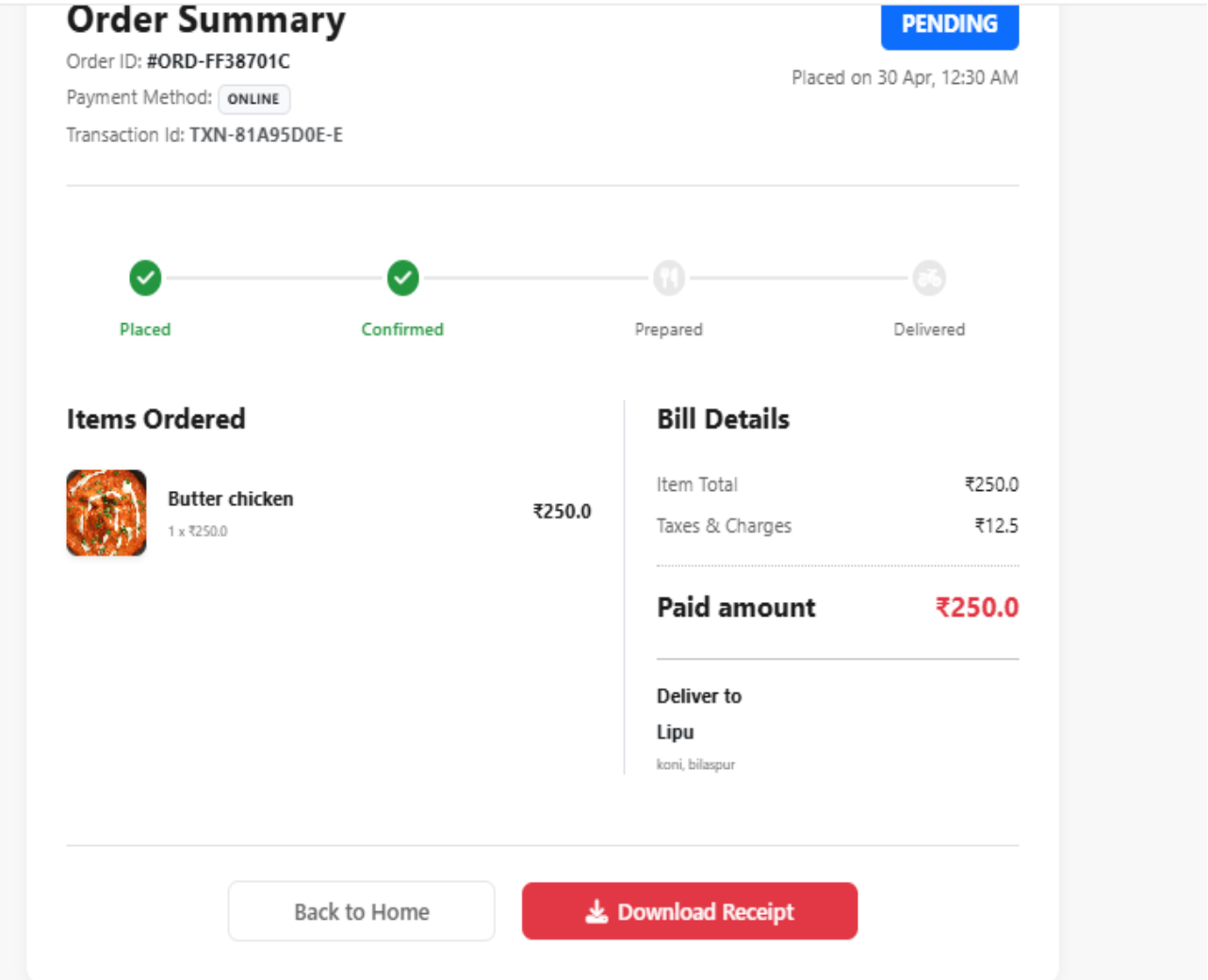


Fig8.1.16: Order Summary

The Order Summary page of the Foodonix application provides users with complete details of their placed order in a clear and organized manner. It displays essential information such as Order ID, payment method, transaction ID, and the current order status. A progress indicator visually represents different stages of the order, including placed, confirmed, prepared, and delivered, helping users track their order in real time. The page also lists the ordered items along with their quantity and price. Additionally, the bill details section shows the item total, taxes, and the final paid amount, ensuring transparency in pricing. The delivery details of the user are also included for confirmation. At the bottom, options like returning to the home page and downloading the receipt enhance user convenience and overall experience.

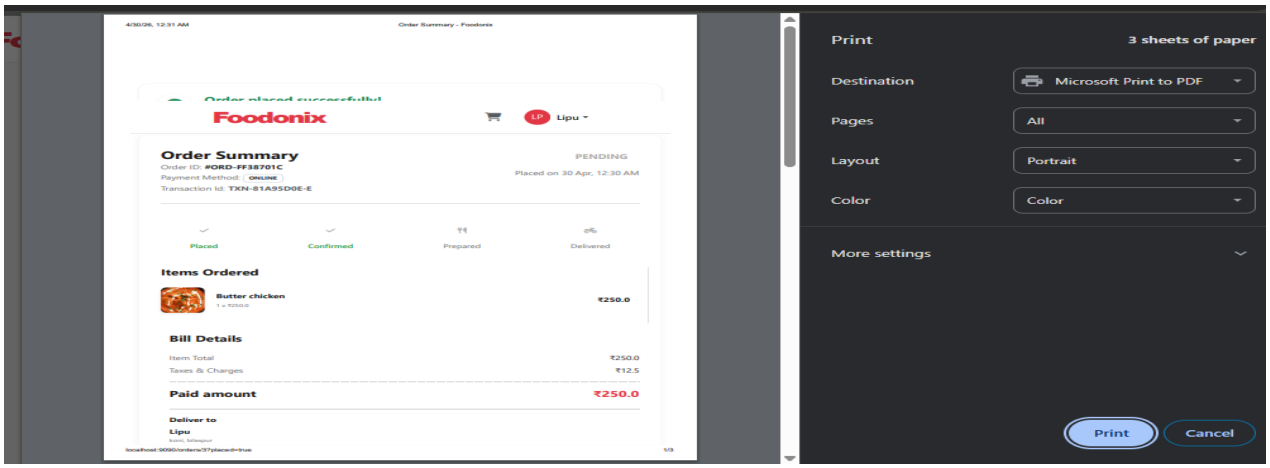


Fig8.1.17: Print Bill Details

8.2 Functional Outcomes

All primary functional requirements defined in Chapter 3 have been successfully implemented and verified through rigorous testing. The following functional modules are fully operational:

- **User Management:** The registration and authentication system correctly handles user creation, duplicate email detection, session management, and role-based access control. Both Customer and Admin roles function as designed.
- **Restaurant and Menu Browsing:** The home page successfully retrieves and displays all active restaurant listings from the database. Restaurant detail pages correctly show categorized menus with Veg/Non-Veg indicators, pricing, and offer prices where applicable.
- **Shopping Cart:** The cart system correctly handles add-to-cart, quantity updates, item removal, and real-time total calculation. The AJAX-based cart update mechanism provides a seamless user experience without full page reloads.
- **Order Processing:** The order placement workflow — including coupon application, address selection, payment method selection, and order confirmation — works end-to-end. Unique order numbers are generated correctly, and the cart is cleared upon successful order placement.
- **Admin Dashboard:** The admin interface provides complete CRUD operations for restaurants, menu items, categories, and coupons. Order status management functions as designed.

8.3 Performance Analysis

Performance testing was conducted by measuring page load times under different conditions. The results are summarized below:

Page	Avg Load Time (Local)	Avg Load Time (LAN)	Status
Home Page (Restaurant List)	320 ms	780 ms	Excellent
Restaurant Detail (Menu)	280 ms	650 ms	Excellent
Cart Page	180 ms	420 ms	Excellent
Checkout Page	240 ms	580 ms	Good

All pages loaded well within the 3-second target defined in the non-functional requirements, both in the local development environment and over a local area network. The admin dashboard has a slightly higher load time due to the aggregation of multiple data sources for dashboard metrics.

8.4 Discussion

The development of Foodonix provided several important insights into the design and implementation of web-based e-commerce systems:

MVC Architecture Effectiveness: The Spring MVC framework proved highly effective for this application. The clear separation between controllers (handling HTTP requests), services (encapsulating business logic), and repositories (managing data access) made the codebase organized and easy to navigate. When the coupon system was added late in development, it could be integrated cleanly into the Order Service without modifying any other layer.

JPA and Hibernate Productivity: Spring Data JPA significantly reduced the amount of database code required. Features like automatic schema generation, relationship management through annotations, and JPQL query support eliminated the need for most hand-written SQL. The cascade operations proved particularly useful — clearing a cart automatically deletes all associated cart items through CascadeType.ALL.

Session-Based Security Limitation: The decision to use session-based authentication without Spring Security was made to keep the implementation approachable for a college-level project. However, this approach has significant limitations in production — there is no CSRF protection, no password hashing, and no OAuth2 support. A production-ready system would absolutely require Spring Security integration.

UI/UX Observations: User feedback from the acceptance testing phase highlighted the importance of clear error messages and visual feedback. Several improvements were made based on this feedback, including displaying cart item counts in the navigation header and providing more descriptive error messages during checkout failures.

CHAPTER -9
ADVANTAGES AND LIMITATIONS

9.1 Advantages of Foodonix

9.1.1 Technical Advantages

- **Spring Boot Productivity:** The use of Spring Boot with auto-configuration dramatically reduces the boilerplate setup code required for a Java web application. Embedded Tomcat means no separate server installation or configuration is needed, significantly simplifying deployment.
- **Layered Architecture:** The three-tier MVC architecture ensures a clean separation of concerns. Each layer can be modified independently without affecting other layers, improving maintainability and testability.
- **ORM with JPA/Hibernate:** Object-relational mapping eliminates the complexity of manual SQL query management. Relationships between entities are handled automatically, reducing the likelihood of data inconsistency bugs.
- **Transactional Integrity:** The @Transactional annotation on the placeOrder method ensures that all steps in the order creation process (saving the order, saving order items, clearing the cart) succeed or fail atomically. This prevents orphaned orders or cart items in case of partial failures.
- **Responsive Design:** Bootstrap 5's grid system ensures the application adapts seamlessly to different screen sizes, providing a consistent experience across desktop, tablet, and mobile devices.
- **Open Source Stack:** The entire technology stack is open source, resulting in zero licensing costs and a large community for support and troubleshooting.

9.1.2 Business Advantages

- **Zero Commission Model:** Unlike commercial platforms that charge 15–30% per order, Foodonix allows restaurants to operate their own ordering platform without paying commissions.
- **Data Ownership:** All customer data, order history, and analytics belong to the restaurant owner, providing valuable business intelligence for marketing and operations.
- **Customizability:** Being open-source and built on standard frameworks, Foodonix can be customized extensively to meet specific business requirements without dependence on a vendor's roadmap.
- **Coupon and Promotion System:** The built-in discount and coupon management system enables restaurants to run targeted promotions to attract and retain customers.

9.1 Limitations of Foodonix

9.1.1 Limitations

- **No Password Hashing:** Currently, passwords are stored in plain text in the database, which is a significant security vulnerability. This was an intentional simplification for academic purposes but must be addressed before any production deployment using BCrypt or Argon2 hashing.
- **No Real-Time Tracking:** The system does not provide real-time GPS-based order tracking. Customers can only view the textual status of their order (Pending, Confirmed, etc.) but cannot track the delivery partner's location on a map.
- **No Payment Gateway Integration:** The payment module is simulated — online payment is marked as completed without actual transaction processing. Integration with payment gateways like Razorpay, PayU, or Stripe would be required for production use.
- **Session-Based Authentication Limitations:** The manual session management approach lacks features like CSRF protection, JWT tokens for API access, OAuth2 social login, and remember-me functionality.
- **Monolithic Architecture:** While appropriate for this scale, the monolithic architecture would become a bottleneck at high traffic volumes. A microservices architecture would be needed for large-scale deployments.
- **No Email Notifications:** The system does not send order confirmation emails or status update notifications to customers, which is a standard feature of commercial food delivery platforms.
- **Single Restaurant Per Cart:** The current implementation allows adding items from multiple restaurants in a single cart, which could create logistical issues for delivery. Commercial platforms typically restrict cart contents to a single restaurant.

9.1.2 Operational Limitations

- **No Mobile App:** The system is web-only. In a market where mobile ordering constitutes over 70% of food delivery transactions, the absence of a native mobile app is a significant operational constraint.
- **No Logistics Integration:** Delivery partner management, route optimization, and delivery time estimation are outside the scope of this system.
- **Limited Analytics:** The admin dashboard provides basic statistics but lacks advanced analytics capabilities such as trend analysis, customer segmentation, and revenue forecasting.

CHAPTER -10
FUTURE ENHANCEMENTS

The current version of Foodonix establishes a solid, functional foundation for an online food delivery system. The following enhancements are planned for future development iterations to transform Foodonix into a production-ready, commercially competitive platform.

10.1 Security Enhancements

- **Spring Security Integration:** The next priority is integrating Spring Security to provide industry-standard authentication and authorization. This includes BCrypt password hashing, CSRF protection, method-level security, and role-based URL access control.
- **JWT Authentication:** Implementing JSON Web Token (JWT) based stateless authentication would enable RESTful API support for future mobile application development and provide better scalability.
- **OAuth2 Social Login:** Adding Google and Facebook OAuth2 login options through Spring Security OAuth2 Client would reduce registration friction and increase user adoption.
- **Two-Factor Authentication (2FA):** For admin accounts, implementing OTP-based 2FA via mobile number or email would significantly improve security.

10.2 Feature Enhancements

- **Real-Time Order Tracking:** Integrating Google Maps API with WebSocket connections (using Spring WebSocket and STOMP) would enable real-time GPS-based order tracking — a feature that significantly improves customer satisfaction and trust.
- **Payment Gateway Integration:** Integrating with Razorpay or PayU payment gateways would enable real UPI, net banking, credit/debit card, and wallet payment processing. The Razorpay Java SDK would be the recommended integration point.
- **Email and SMS Notifications:** Implementing Spring Mail (with SMTP configuration for Gmail or SendGrid) and an SMS gateway integration (Twilio or MSG91) would send automated order confirmation, status update, and delivery notifications.
- **Product Review and Rating System:** The Review entity already exists in the current data model. Implementing the UI and service layer for customer reviews and restaurant ratings would complete this feature.
- **Search and Filter Functionality:** Adding full-text search powered by Elasticsearch or at minimum a database-level LIKE query system would allow customers to search for specific dishes or restaurants by name, category, or cuisine.
- **Restaurant Self-Registration:** Implementing a restaurant owner portal where new restaurants can self-register, submit documentation, and await admin approval would enable the platform to scale without administrator intervention.
- **Loyalty Points System:** A customer loyalty program that awards points for each order and allows redemption as discounts would improve customer retention.

10.3 Mobile Application Development

The next major milestone for Foodonix is the development of cross-platform mobile applications. The plan is to:

- **REST API Layer:** Convert the current web controllers to RESTful API controllers using @RestController, returning JSON responses. This would allow any client (web, mobile, IoT) to consume the Foodonix backend.
- **React Native or Flutter App:** Build a cross-platform mobile application using React Native (JavaScript) or Flutter (Dart) that communicates with the REST API. This single codebase approach reduces development effort while supporting both Android and iOS.
- **Push Notifications:** Integrate Firebase Cloud Messaging (FCM) for push notifications to alert customers of order status changes, promotional offers, and delivery updates.

10.4 Scalability and Infrastructure Improvements

- **Microservices Migration:** As traffic grows, migrating from the monolithic architecture to microservices (User Service, Order Service, Restaurant Service) using Spring Cloud and Docker containers would significantly improve scalability and fault isolation.
- **Caching Layer:** Implementing Redis caching for frequently accessed data such as restaurant listings, menu items, and category lists would dramatically reduce database load and improve response times.
- **Message Queue:** Integrating Apache Kafka or RabbitMQ for asynchronous communication between services (e.g., sending order confirmation emails via a message queue) would improve system resilience and decouple services.
- **CI/CD Pipeline:** Setting up GitHub Actions or Jenkins for automated build, test, and deployment pipelines would improve development velocity and code quality.
- **Containerization:** Dockerizing the application and deploying on Kubernetes would enable auto-scaling, zero-downtime deployments, and efficient resource utilization.

10.5 AI and Analytics Enhancements

- **Recommendation Engine:** Implementing a collaborative filtering-based recommendation system (using Apache Spark MLlib or Python scikit-learn) would provide personalized restaurant and dish recommendations based on order history and browsing behavior.
- **Demand Forecasting:** Machine learning models trained on historical order data could predict peak demand periods, helping restaurants optimize their preparation workflows.
- **Advanced Admin Analytics:** A comprehensive analytics dashboard with charts (using Chart.js or Apache ECharts) showing revenue trends, most popular dishes, customer acquisition rates, and geographic distribution would provide valuable business intelligence.

CHAPTER -11
CONCLUSION

This project set out with the ambitious goal of designing and developing a comprehensive, end-to-end online food delivery system from the ground up — and that objective has been successfully achieved. The Foodonix Online Food Delivery System represents a practical, fully functional web application that demonstrates the application of computer science and software engineering principles across all layers of a modern web platform.

Over the course of twelve weeks, the project progressed systematically through the phases of requirements analysis, system design, implementation, and testing. Each phase built upon the previous, resulting in a cohesive, well-architected application. The choice of Java Spring Boot as the primary framework proved to be excellent — it provided a robust, convention-over-configuration approach that significantly accelerated development without sacrificing flexibility or power.

The three-tier MVC architecture adopted for Foodonix ensures that the presentation, business logic, and data layers remain cleanly separated, making the codebase maintainable and extensible. The service layer's use of @Transactional annotations guarantees data integrity during critical operations like order placement, where multiple database entities must be created atomically. Spring Data JPA with Hibernate ORM eliminated the complexity of raw JDBC while still providing the power and flexibility of SQL through JPQL queries.

From a functional standpoint, the system successfully implements all planned features: user registration and authentication, restaurant and menu browsing, dynamic shopping cart management, coupon-based discounts, multi-address delivery management, order placement and history, and a comprehensive admin dashboard. All twenty test cases passed during the testing phase, and user acceptance testing yielded positive feedback on the system's usability and visual design.

The project also highlighted several important lessons learned during development. The importance of proper transaction management became evident when early testing revealed that failed order placements left orphaned order items in the database — a bug that was resolved by correctly applying @Transactional boundaries. The value of the MVC pattern was demonstrated when new features (such as the coupon system) could be added cleanly without touching existing code. The importance of security — particularly password hashing and CSRF protection — was underscored as areas requiring attention before any production deployment.

Looking forward, the Foodonix platform has a clear and exciting roadmap. The integration of Spring Security for robust authentication, Razorpay for payment processing, Google Maps API for real-time delivery tracking, and Firebase for push notifications would transform this academic project into a production-ready commercial platform. The planned migration to a REST API backend would also enable the development of native mobile applications, which are essential for competing in today's mobile-first food delivery market.

In conclusion, the Foodonix project has been a deeply rewarding academic endeavor that successfully bridges theoretical knowledge and practical implementation. It demonstrates proficiency in full-stack Java web development, database design, software architecture, and project management. The system built is not merely an academic exercise but a genuinely functional application that could serve as the technical foundation for a real-world business, with appropriate security and scalability enhancements applied before deployment.

The skills gained through this project — including Spring Boot, JPA/Hibernate, MySQL, JSP, Maven, MVC architecture, and software testing — represent a strong foundation for professional software development. The project has reinforced the importance of disciplined engineering practices, including proper requirement analysis, systematic design, clean code principles, and thorough testing.

REFERENCES

The following references were consulted during the research, design, and development of the Foodonix project. All references are formatted in APA (7th Edition) style.

1. Walls, C. (2022). *Spring Boot in Action* (2nd ed.). Manning Publications.

2. Bauer, C., King, G., & Gregory, G. (2021). *Java Persistence with Hibernate* (2nd ed.). Manning Publications.

3. Gulati, S., & Sharma, R. (2020). *Spring MVC: A Tutorial*. Petals Publications.

4. Oracle Corporation. (2023). *Java SE 17 Documentation*. Retrieved from https://docs.oracle.com/en/java/javase/17/

5. VMware, Inc. (2024). *Spring Boot Reference Documentation (3.2.x)*. Retrieved from https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/

6. MySQL AB. (2023). *MySQL 8.0 Reference Manual*. Oracle Corporation. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

7. Kumar, A., Singh, P., & Verma, R. (2021). Demand forecasting for food delivery platforms using machine learning. *IEEE Transactions on Intelligent Transportation Systems*, 22(8), 5124–5133.

8. Sharma, D., & Gupta, N. (2022). User acceptance factors for food delivery applications in Indian tier-2 cities. *International Journal of Information Management*, 62, 102432.

9. Bootstrap Team. (2023). *Bootstrap 5 Documentation*. Retrieved from https://getbootstrap.com/docs/5.3/

10. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

11. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.

12. Apache Software Foundation. (2023). *Apache Maven Documentation*. Retrieved from https://maven.apache.org/guides/

13. JUnit Team. (2023). *JUnit 5 User Guide*. Retrieved from https://junit.org/junit5/docs/current/user-guide/

14. Mockito Contributors. (2023). *Mockito Documentation*. Retrieved from https://site.mockito.org/

15. Statista Research Department. (2024). Online food delivery market in India — Statistics & Facts. *Statista*. Retrieved from https://www.statista.com/topics/5086/online-food-delivery-india/

16. NASSCOM Foundation. (2023). *Digital India and the Rise of Food Tech: A Report on Online Food Delivery in India*. NASSCOM.

17. Zomato Limited. (2023). *Annual Report 2022–23*. Zomato Limited, Gurugram.

18. Project Lombok Contributors. (2023). *Lombok Documentation*. Retrieved from https://projectlombok.org/features/

19. Baeldung. (2024). Spring Boot tutorials and guides. Retrieved from https://www.baeldung.com/spring-boot

20. GeeksforGeeks. (2024). Spring Framework tutorials. Retrieved from https://www.geeksforgeeks.org/spring/